

Integration Methods in cppDE

Simon Beyer

2026-09-19

Contents

1	Overview	3
2	Code generation	3
3	cppODE()	4
3.1	Initial value problem and four methods	4
3.2	Stiffness and the choice of method	4
3.3	Multistep methods	5
3.4	Single-step methods	6
3.5	Adaptive step-size control	8
3.6	Dense output	8
3.7	Forward sensitivities	9
3.8	Events and restarts	11
3.9	Reverse-mode sensitivities: the discrete adjoint	14
3.10	Batches of conditions	22
4	cvode()	23
4.1	What is shared	23
4.2	What differs	23
4.3	Adjoint sensitivity analysis	23
5	cppFUN()	24
5.1	Entry points and derivative directions	25
5.2	Tangent, hessian, cotangent and curvature	25
5.3	Pass-through of unmodelled inputs	26
6	Benchmarks	27
	References	30
A	BDF and NDF coefficients	31
B	Adams-Moulton coefficients	31
C	Rosenbrock4 coefficients	31
C.1	Diagonal coefficient, time partials, and node positions	32
C.2	Stage couplings α_{ij}	32
C.3	Stage couplings γ_{ij}	32
C.4	Solution and error weights	32
C.5	Dense-output coefficients	33

1 Overview

cppDE generates and compiles per-model C++ code for three related problems through three entry points that share a common R interface: `cppODE()`, `cvode()`, and `cppFUN()`. Table 1 summarises their scope.

	<code>cppODE()</code>	<code>cvode()</code>	<code>cppFUN()</code>
Problem class	ODE	ODE	algebraic $y = g(x, p)$
Methods	<code>bdf</code> , <code>adams</code> , <code>rb4</code> , <code>tsit5</code>	<code>bdf</code> , <code>adams</code>	
First-order sensitivities	in-tree dual-number AD	SUNDIALS, generated JVP	dual AD
Reverse-mode gradient	yes (discrete adjoint)	yes (CVODES ASA)	yes (generated VJP)
Second-order sensitivities	yes (nested duals, fwd-over-rev)	no	yes (nested duals)
External dependencies	LAPACK; KLU optional	SUNDIALS ≥ 6.0 ; KLU opt.	none
Codegen	graph AD + R CMD SHLIB	graph AD + R CMD SHLIB	graph AD + R CMD SHLIB

Table 1: The three cppDE entry points and what they cover. The codegen pipeline of Section 2 (parse the user’s expressions into an expression graph, differentiate on the graph, emit C++, compile via R CMD SHLIB, load via `dyn.load`) is shared, as is the `solveODE()` dispatcher used by both integrator backends.

The integrator entry points `cppODE()` and `cvode()` expose the same R-side interface: the user supplies named character vectors of right-hand sides together with optional events, forcings, and parameter vectors, and the returned model handle is passed to `solveODE()`. The two backends differ only in the numerical core. `cppODE()` emits a self-contained C++ source that includes the headers under `inst/include/cppde/`; the resulting binary depends on LAPACK and, optionally, KLU. `cvode()` emits a thin glue layer that links against the system-installed SUNDIALS CVODE/CVODES library and inherits its numerical behaviour.

Section 2 describes the code generator the three entry points share. Section 3 describes the in-tree integrator `cppODE()`: its four time-stepping methods, forward-mode automatic differentiation for parameter sensitivities, the reverse-mode discrete adjoint, whose cost does not depend on the number of parameters, and the transport of sensitivities across events. Section 4 describes the SUNDIALS-backed `cvode()` integrator and its adjoint sensitivity analysis. Section 5 describes `cppFUN()`, which compiles algebraic maps $y = g(x, p)$ with their first and second derivatives and no time integration, typically observation maps and reparametrisations $p = \Phi(\theta)$. Section 6 benchmarks the two integrator backends against each other on 43 problems, and the reverse and second-order modes against their forward counterparts.

2 Code generation

Every entry point turns the user’s expressions into straight-line C++ templated over the scalar type, so that one source serves the value, the dual-number and the nested-dual instantiation. The derivatives the integrators need beyond that, namely the Jacobian, the contractions of the discrete adjoint, and the time derivatives of the right-hand side and of the event roots, are formed at code-generation time by automatic differentiation on an expression graph.

Expression graph. Each expression is read by Python’s own parser and lowered into a directed acyclic graph whose nodes are hash-consed: a node is identified by its operation, its operands and its constant attributes, and is created at most once. A subexpression that occurs several times is therefore a single node, without a separate elimination pass. Sums and products are kept flat with numeric coefficients and exponents, and numeric constants are normalised so that equal values map to equal nodes. Syntax the parser does not read is passed to SymPy and translated into the same graph; SymPy also supplies the derivative of a function the graph has no rule for.

Derivatives. A forward sweep over the graph forms a Jacobian-vector product, a reverse sweep a vector-Jacobian product; both add nodes to the same graph, so derivative and value share their common parts. The Jacobian $J = \partial f / \partial x$ is assembled in one of two ways. By entries, each state-dependent node carries the sparse gradient of its value with respect to the states, and the gradients of the roots are the nonzero entries of J . The work of this accumulation is $W_{\text{ent}} = \sum_v |\text{supp } \nabla v|$, summed over the E_s state-dependent nodes

v of f . Entries are used while $W_{\text{ent}} \leq \max(4E_s, 50\,000)$. Beyond that, a greedy distance-2 colouring of the column intersection graph groups structurally orthogonal columns, and if it needs at most $n_x/4$ colours, one Jacobian-vector product per colour, seeded with ones on the columns of that colour, recovers all entries of those columns; otherwise entries are kept. The reverse contractions $J^\top \lambda$ and $(\partial f / \partial p)^\top \lambda$ are vector-Jacobian products, and the derivatives the Rosenbrock stages and the event localisation require are Jacobian-vector products along $\dot{x} = f$. A single forward or reverse sweep costs time linear in the size of the graph. The entry accumulation costs W_{ent} , which grows with the density of J , and the colouring path costs one sweep per colour.

Linear tables. A sum of at least twelve terms $c_j x_j \varphi$ with numeric c_j and a common state-free factor φ becomes a row of a constant matrix C , so that $f(x) = \phi(x, Cx)$. The generated code evaluates $z = Cx$ once per call from a sparse or dense table, and assembles the Jacobian as

$$J = \frac{\partial \phi}{\partial x} + \frac{\partial \phi}{\partial z} C, \quad (1)$$

while the reverse contractions apply $C^\top$. A dense coupling, such as a dipolar interaction among all states, then costs a table rather than a sum of expressions per entry.

Loops. A method-of-lines discretisation repeats the same statement for every grid point. Statements whose graphs agree up to state indices and constants form a class; a class of at least sixteen instances is emitted as one loop over index and constant tables, and a sum of many terms of one structure as an inner loop. Differentiation remains per instance, so the work of the generator grows with the number of instances, while the size of the emitted code for a regular model does not.

3 cppODE()

3.1 Initial value problem and four methods

This section describes the in-tree integrator of `cppODE()`: the four time-stepping methods, their step-size control and dense output, forward sensitivities through dual numbers, time- and root-triggered events, and the reverse mode. The coefficients of each method are listed in Appendices A to D.

The initial-value problem solved by `cppODE()` is

$$\dot{x} = f(x, p_{\text{dyn}}, t), \quad x(t_0) = p_{\text{init}}, \quad (2)$$

with state $x \in \mathbb{R}^{n_x}$ and parameter vector $p = (p_{\text{init}}, p_{\text{dyn}})$ collecting the initial conditions and the dynamic parameters. The right-hand side f may depend explicitly on time, in which case the system is non-autonomous; an explicit time argument is also necessary for the forcing-function machinery used to inject time-dependent inputs. The Jacobian of f with respect to the state is denoted $J = \partial f / \partial x$ throughout.

3.2 Stiffness and the choice of method

Stability is analysed on the linear test equation $\dot{y} = \lambda y$ with $\lambda \in \mathbb{C}^-$, whose exact solution decays. A one-step method applied to it multiplies y by an amplification factor $R(z)$ per step, $z = h\lambda$. The method is *A-stable* if $|R(z)| \leq 1$ on the entire left half-plane, so that the numerical solution decays whenever the exact one does, for any step size; *A(α)-stable* if the same bound holds on the wedge $\{z \in \mathbb{C} : |\arg(-z)| \leq \alpha\}$ for some angle $\alpha < 90^\circ$; and *L-stable* if it is A-stable and $R(z) \rightarrow 0$ as $\text{Re}(z) \rightarrow -\infty$, which damps very stiff transients within one step.

The initial-value problem (2) is *stiff* on $[t_0, t_{\text{end}}]$ when an explicit method has to take steps much smaller than local accuracy demands, to keep the products $h\lambda_k$ inside its stability domain, where $\{\lambda_k\}$ are the eigenvalues of J along the trajectory. Stiffness therefore depends on the spectrum of J , on the integration horizon, and on the requested tolerance. Chemical-kinetics networks with rate constants spanning several orders of magnitude are stiff in this sense, as are many discretised parabolic PDEs.

BDF/NDF and Rosenbrock4 have stability domains that contain the left half-plane or a wide wedge of it and are suited to stiff problems; Adams-Moulton and Tsit5 are not, whatever the tolerance. The cost of a wrong choice is asymmetric: an explicit method on a stiff problem fails or is driven to uneconomic step sizes, while an implicit method on a non-stiff problem pays a constant overhead per step. BDF/NDF is therefore the default when the spectrum of J is unknown, and for a large model with a sparse Jacobian it runs on the sparse linear solver (`sparse = TRUE`).

3.3 Multistep methods

Linear multistep methods build the solution at the new time step x_n from a linear combination of the most recent $k + 1$ states and right-hand-side values, $\{x_{n-j}, f_{n-j}\}_{j=0}^k$. Reusing earlier steps, they attain high accuracy with few right-hand-side evaluations per step.

cppDE implements the BDF and the Adams families in the Nordsieck representation used throughout SUNDIALS [1]. Instead of the past $k + 1$ states it stores the *scaled derivative array*

$$z_n = \left(x_n, h_n \dot{x}_n, \frac{h_n^2}{2!} \ddot{x}_n, \dots, \frac{h_n^k}{k!} x_n^{(k)} \right),$$

the value at the current time and its first k derivatives, each scaled by the matching power of the step size. A change of step size or order is then a linear transformation of z_n rather than a refit through past data.

Backward Differentiation Formula (bdf)

The BDF formula of order q defines x_n implicitly by the condition

$$\sum_{j=0}^q \alpha_j x_{n-j} = h_n f(x_n, t_n), \quad (3)$$

which is exact whenever $x(t)$ is a polynomial of degree at most q on the past $q + 1$ nodes. The method is A-stable for $q \leq 2$ and A(α)-stable for $q \in \{3, 4, 5\}$; the stability angle α shrinks monotonically from 86° at $q = 3$ to 51° at $q = 5$.

The implicit corrector (3) is solved by Newton iteration,

$$W \Delta x^{(\nu)} = -\tilde{r}(x_n^{(\nu)}), \quad x_n^{(\nu+1)} = x_n^{(\nu)} + \Delta x^{(\nu)}, \quad (4)$$

with leading coefficient $\beta_0 = 1/\alpha_0$ and the iteration matrix

$$W = \frac{1}{h_n \beta_0} \mathbb{1} - J(x_n^{(\nu)}, t_n), \quad (5)$$

in the form adopted by SUNDIALS [1], which differs from the textbook expression $I - h_n \beta_0 J$ by the scalar factor $h_n \beta_0$. The factorisation of W is the dominant cost of a step and is reused across steps, as described below.

The default in cppDE is the Numerical Differentiation Formula (NDF) variant of [2], which adds a correction proportional to the Nordsieck predictor error to the BDF corrector,

$$\sum_{m=1}^q \frac{1}{m} \nabla^m x_n - h_n f(x_n, t_n) - \kappa_q \gamma_q (x_n - x_n^{\text{pred}}) = 0, \quad \gamma_q = \sum_{m=1}^q \frac{1}{m}, \quad (6)$$

where ∇ denotes the backward-difference operator. The modification scales β_0 in (4) by $(1 - \kappa_q)$ and multiplies the leading truncation-error coefficient $1/(q + 1)$ by $1 + \kappa_q(q + 1)\gamma_q$. With $\kappa_q < 0$ the truncation error at a fixed step size decreases, which permits a larger step at the same accuracy. The values of κ_q are tabulated in Appendix A; they give the step-size gains over classical BDF reported by [2], 26 % for $q \leq 3$ and 12 % for $q = 4$, at a small reduction in stability angle. Setting `useNDF = FALSE` recovers the classical BDF formulas ($\kappa_q \equiv 0$).

The corrector takes at most three iterations, each with one right-hand-side evaluation and one linear solve. It converges when the damped correction norm $\delta \min(1, \rho)/\tau_q \leq 1$, where δ is the WRMS norm of the latest correction, ρ the estimated contraction rate and τ_q an order-dependent constant. As in CVODE, W is refactorised when $\gamma = h_n \beta_0$ has moved by more than 30 % since the last factorisation, after 20 steps, or after a failed corrector; the Jacobian itself is re-evaluated after 51 steps or when a failure is attributed to it. A sparse W then changes only on its diagonal. BDF/NDF is the default method in cppDE.

Adams-Moulton in PECE form (adams)

The Adams-Moulton family integrates the local interpolant of f over $[t_{n-1}, t_n]$. Writing f as the unique polynomial through the $q + 1$ samples $\{(t_{n-j}, f_{n-j})\}_{j=0}^q$ and integrating gives

$$x_n = x_{n-1} + h_n \sum_{j=0}^q \beta_j f(x_{n-j}, t_{n-j}), \quad (7)$$

which is implicit in x_n through the $j = 0$ term. cppDE solves the corrector by fixed-point iteration in predict-evaluate-correct-evaluate (PECE) form, without a Jacobian or a linear solve. The Nordsieck predictor supplies x_n^P , the right-hand side is evaluated there, and (7) is applied with $f(x_n^P, t_n)$ in place of $f(x_n, t_n)$. If the correction has not converged, the evaluation and the correction are repeated, at most three times in all, as in CVODE. A step that converges on the first correction costs two right-hand-side evaluations.

The Adams-Moulton stability domain on the negative real axis shrinks rapidly with order; for $q \geq 3$ it contains no wedge of $\mathbb{C}^-$, so the method is unsuitable for stiff problems. For non-stiff problems with smooth solutions and a long integration horizon, its maximum order $q_{\max} = 12$ and two right-hand-side evaluations per step make it competitive with explicit Runge-Kutta methods of fixed order.

3.4 Single-step methods

Single-step methods carry no information beyond the current state and reach order $p > 1$ through internal stages. Both methods below therefore restart after an event at full order, whereas the multistep methods rebuild the Nordsieck history from order 1.

Rosenbrock (rb4)

A Rosenbrock method of s stages with diagonal coefficient γ linearises the implicit Runge-Kutta condition about (x_n, t_n) and defines stage values k_i by

$$Wk_i = f \left(x_n + \sum_{j<i} \alpha_{ij} k_j, t_n + c_i h_n \right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma} k_j + h_n d_i \frac{\partial f}{\partial t}(x_n, t_n), \quad (8)$$

with the iteration matrix

$$W = \frac{1}{\gamma h_n} \mathbb{1} - J(x_n, t_n). \quad (9)$$

The method coefficients are listed in Table 2.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + \sum_{i=1}^s m_i k_i, \quad \hat{x}_{n+1} = x_n + \sum_{i=1}^s \hat{m}_i k_i.$$

Each stage (8) is a single linear solve, with no Newton iteration. Per accepted step cppDE evaluates J and factorises W once, performs five right-hand-side evaluations, and back-substitutes six times.

cppDE uses the order-4 coefficients of [3, Sec. IV.7], the same set as the `rosenbrock4` stepper of Boost.NumericOdeint. The pair is L-stable and carries an embedded order-3 estimator. Dense output uses an order-3 interpolant built from the stage values k_i and the coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$ of Appendix C.

Symbol	Meaning
α_{ij}	weights with which stage i combines all earlier stages k_j , $j < i$
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} \alpha_{ij}$
γ_{ij}	weights with which the linear-solve correction at stage i uses earlier stages
γ	common diagonal coefficient appearing in the iteration matrix W
d_i	weights that pick up the explicit time dependence of f via $\partial f / \partial t$
m_i	weights combining the stages into the advanced solution x_{n+1}
$\hat{m}_i$	weights combining the stages into the embedded order-3 estimator used for step control

Table 2: Rosenbrock method coefficients.

Rosenbrock methods are competitive with BDF on stiff problems whose Jacobian is cheap relative to the right-hand side, and they restart after an event without rebuilding an order. Rosenbrock4 is the stiff method of choice for systems with frequent state discontinuities at moderate state dimension.

Tsitouras (tsit5)

Tsitouras is the explicit Runge-Kutta pair of order 5(4) derived by [4] under the first column simplifying assumption alone,

$$\sum_{i=1}^s b_i a_{ij} = b_j (1 - c_j), \quad j = 1, \dots, s,$$

without the row simplifying assumption $\sum_j a_{ij} c_j = c_i^2/2$, $i \geq 3$, on which the Dormand-Prince pair rests. Dropping the row condition leaves free parameters, which Tsitouras spends on a smaller leading-error norm; the coefficients of Appendix D satisfy the column condition to rounding and violate the row condition. The pair has $s = 7$ stages with the First-Same-As-Last (FSAL) property,

$$k_7 = f(x_{n+1}, t_n + h_n) = k_1 \text{ at step } n + 1,$$

so an accepted step costs six right-hand-side evaluations rather than seven. The method coefficients are listed in Table 3.

Symbol	Meaning
a_{ij}	weights with which stage i combines earlier stages (strictly lower triangular)
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} a_{ij}$
b_i	weights combining the stages into the advanced order-5 solution; $b_7 = 0$
$\hat{b}_i$	weights combining the stages into the embedded order-4 estimator; $\hat{b}_7 \neq 0$
k_i	right-hand side at stage i : $k_i = f(x_n + h_n \sum_{j<i} a_{ij} k_j, t_n + c_i h_n)$

Table 3: Tsit5 method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + h_n \sum_{i=1}^7 b_i k_i, \quad \hat{x}_{n+1} = x_n + h_n \sum_{i=1}^7 \hat{b}_i k_i,$$

and the local error estimate used for step-size control is

$$e_{n+1} = h_n \sum_{i=1}^7 (\hat{b}_i - b_i) k_i.$$

Dense output uses the step endpoints and their derivatives $(f_n, f_{n+1}) = (k_1, k_7)$, which the FSAL property already holds.

Tsit5 needs no Jacobian and no linear solve, and of the four methods it advances furthest per right-hand-side evaluation. It suits non-stiff problems whose right-hand side is expensive to evaluate.

3.5 Adaptive step-size control

The error estimate e_n delivered by each method is reduced to a scalar through the weighted root-mean-square norm

$$\|e_n\|_{\text{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{j=1}^{n_x} \left(\frac{e_{n,j}}{w_{n,j}} \right)^2}, \quad w_{n,j} = \text{atol} + \text{rtol} |x_{n,j}^{\text{ref}}|, \quad (10)$$

in which $w_{n,j}$ is a per-component tolerance combining an absolute floor **atol** and a relative scale **rtol** $|x_{n,j}^{\text{ref}}|$, so that components of different magnitude and unit enter on a common scale. The reference state x_n^{ref} that enters the relative term is the accepted solution $|x_n|$ at the start of the step in BDF/NDF and Adams, as in CVODE, so the weights are fixed for the whole step, and the component-wise maximum $\max(|x_n|, |x_{n+1}|)$ in the single-step methods. A step is accepted when $\|e_n\|_{\text{WRMS}} \leq 1$.

The single-step methods select the next step size with the proportional-integral (PI) controller of [3]. Write $\varepsilon_n = \|e_n\|_{\text{WRMS}}$ and let p be the order of the embedded error estimate, $p = 3$ for Rosenbrock4 and $p = 4$ for Tsit5. After an accepted step

$$h_{n+1} = h_n \min \left\{ \rho_{\max}, \max \left(\rho_{\min}, S \varepsilon_n^{-(\alpha+\beta)} \varepsilon_{n-1}^\beta \right) \right\}, \quad \alpha = \frac{0.7}{p+1}, \quad \beta = \frac{0.4}{p+1},$$

with $S = 0.9$, $\rho_{\min} = 0.2$, $\rho_{\max} = 5$, and ε_{n-1} the norm of the previous accepted step, floored at 0.01. On the first step and after a rejection the integral form $S \varepsilon_n^{-1/(p+1)}$ replaces the PI factor, and after a rejection the factor is further capped at 1. A rejected step is retried with the factor $S \varepsilon_n^{-1/(p+1)}$ clamped to $[\rho_{\min}, 0.9]$.

The multistep methods select step size and order as CVODE does. With $d_{q'}$ the WRMS norm of the error estimate at order q' , the candidate ratios are

$$\eta_{q'} = \frac{1}{(B_{q'} d_{q'})^{1/(q'+1)} + 10^{-6}}, \quad q' \in \{q-1, q, q+1\},$$

with $B_{q-1} = B_q = 6$ and $B_{q+1} = 10$. The neighbouring orders are compared $q+1$ steps after a change to order q and every second step while the order is unchanged, and the largest ratio fixes the next order. A ratio below 1.5 leaves h unchanged, and the ratio is capped at 10^4 on the first step and at 10 afterwards.

The first step

The first step has no previous error norm. cppDE estimates it from the initial state as in [5], Algorithm II.4.4: a rough $h_0 \sim \|x_0\|/\|f_0\|$ fixes a scale, a second derivative $\ddot{x}$ along the trajectory fixes the curvature, and

$$h_1 = \left(\frac{0.01}{\max(\|f_0\|, \|\ddot{x}\|)} \right)^{1/(p+1)} \quad (11)$$

turns the two into a step for a method of order p . The implicit methods obtain $\ddot{x} = \partial f / \partial t + Jf$ from the generated Jacobian, the explicit one from a finite difference of f . All norms are the weighted norms of (10); with **sensErrCon** = **TRUE** they include the sensitivity components.

Two bounds follow, in the form CVODE uses. An upper bound keeps a single Euler step from changing any component by more than a fraction of its own scale, and is capped by the remaining interval; a lower bound keeps h_0 above the rounding of the time variable. The multistep methods use a part of the CVODES routine **cvHin**, which refines $\sqrt{h_{\text{lb}} h_{\text{ub}}}$ by a finite-difference curvature until two successive estimates agree within a factor of two, so the first step equals the one CVODES takes on the same problem. The estimator runs again after every event, on the post-event state.

3.6 Dense output

User-requested output times do not in general coincide with the adaptive step grid produced by the controller. Each accepted step therefore carries a local interpolant that allows evaluation of $x(t)$ at any time inside that

step at the cost of polynomial arithmetic only, and no right-hand-side evaluation. With $s = (t - t_n)/h_n \in [0, 1]$ the three families use:

- Tsit5: the cubic Hermite interpolant through x_n, x_{n+1} and the FSAL derivatives k_1, k_7 ,

$$x(t) = (1 + 2s)(1 - s)^2 x_n + s^2(3 - 2s) x_{n+1} + s(1 - s)^2 h_n k_1 + s^2(s - 1) h_n k_7.$$

- Rosenbrock4: the stage-based cubic of [3],

$$x(t) = (1 - s) x_n + s(x_{n+1} + (1 - s)(c_3 + s c_4)), \quad c_3 = \sum_{i=1}^5 d_i^{(2)} k_i, \quad c_4 = \sum_{i=1}^5 d_i^{(3)} k_i,$$

with the coefficients of Table 11.

- BDF/NDF and Adams: the Nordsieck polynomial of degree q held after the step, $x(t) = \sum_{j=0}^q z_{n+1,j} ((t - t_{n+1})/h_n)^j$.

Both single-step interpolants are of order 3, below the order of the step itself (5 for Tsit5, 4 for Rosenbrock4); the multistep interpolant has the order q of the step.

3.7 Forward sensitivities

Dual-number arithmetic

Parameter sensitivities of the trajectory are computed by forward-mode automatic differentiation rather than by finite differences, whose truncation and round-off errors need several integrations to control. Forward-mode AD augments every floating-point scalar with a component that carries its directional derivative. Arithmetic operations and elementary functions are overloaded for the augmented type, and the C++ source that evaluates f is compiled unchanged for both the primal and the sensitivity computation.

The augmented type is the *dual number*

$$\tilde{x} = x + x' \varepsilon, \quad \varepsilon^2 = 0, \quad \varepsilon \neq 0,$$

where $x \in \mathbb{R}$ is the primal value and $x' \in \mathbb{R}$ is the directional derivative. Reading $\tilde{x}$ as the first two terms of a Taylor expansion in ε , the arithmetic follows from the rules of calculus: for $\tilde{x} = x + x' \varepsilon$ and $\tilde{y} = y + y' \varepsilon$,

$$\tilde{x} + \tilde{y} = (x + y) + (x' + y') \varepsilon, \quad \tilde{x} \cdot \tilde{y} = xy + (xy' + x'y) \varepsilon,$$

and for any smooth scalar function g ,

$$g(\tilde{x}) = g(x) + g'(x) x' \varepsilon.$$

A composition of these operations carries the derivative of the composed function, so the source that evaluates f over $\mathbb{R}$ returns f and its directional derivative when called with dual inputs.

Tangent storage

A dual carries its value inline and m tangent slots behind a pointer, and m is `ncol` of the tangent, read at run time rather than fixed at compile time. The tangents of temporaries come from a thread-local bump arena, whose allocations since the start of a generated model body are released in one operation when that body returns. The state vectors the stepper keeps across steps instead point into one contiguous $n_x \times m$ block per vector, the tangent slab, sized once per solve. A parameter set wider than one call should hold is answered in blocks of directions; since the step sequence of a tangent-blind solve does not depend on m , the blocks share one grid and the assembled Hessian equals a single pass exactly.

A dual whose width has never been set has no slots, and reading a tangent from it yields zero. Buffers that are zeroed before use, such as the cotangents of a backward sweep, therefore have their width declared first; otherwise their derivatives would be structurally zero.

Sensitivity equations via dual evaluation

Let the ODE right-hand side depend on a parameter vector $p \in \mathbb{R}^{n_p}$,

$$\dot{x} = f(x, p, t), \quad x(t_0) = x_0(p),$$

and define the *sensitivity matrix*

$$S_p = \frac{\partial x}{\partial p} \in \mathbb{R}^{n_x \times n_p}, \quad (S_p)_{ij} = \frac{\partial x_i}{\partial p_j}.$$

Seeding each state component with its row of S_p gives the dual-valued state vector

$$\tilde{x}_i = x_i + (S_p)_{ij} \varepsilon_j,$$

where ε_j is the dual unit associated with direction p_j and summation over j is implied. Evaluating f component-wise over the dual algebra and collecting the ε_j -coefficient of the i -th output gives

$$[f(\tilde{x}, p, t)]_{i, \varepsilon_j} = \frac{\partial f_i}{\partial x_k} (S_p)_{kj} + \frac{\partial f_i}{\partial p_j},$$

where repeated k is summed. Since the primal part of the dual evaluation recovers $\dot{x}_i = f_i(x, p, t)$, differentiating $\dot{x}_i = f_i$ with respect to p_j and identifying the ε_j -coefficient with $(\dot{S}_p)_{ij}$ yields

$$\dot{S}_p = \frac{\partial f}{\partial x} S_p + \frac{\partial f}{\partial p}, \quad S_p(t_0) = \frac{\partial x_0}{\partial p}. \quad (12)$$

One dual evaluation of f therefore yields the right-hand side of (12) for all columns of S_p , at a cost proportional to n_p and without forming $\partial f / \partial x$.

Seeding: from S_p to S_θ

The parameters of inference are often not the entries of p but a vector $\theta \in \mathbb{R}^M$ to which the dynamic parameters p_{dyn} and the initial condition $x_0 = p_{\text{init}}(\theta)$ are linked through a reparametrisation $p = \Phi(\theta)$. The quantity of interest is then

$$S_\theta = \frac{\partial x}{\partial \theta} \in \mathbb{R}^{n_x \times M},$$

related to S_p by the chain rule $S_\theta = S_p \partial p / \partial \theta$. Substituting this relation into (12) yields the evolution equation for S_θ directly,

$$\dot{S}_\theta = \frac{\partial f}{\partial x} S_\theta + \frac{\partial f}{\partial p_{\text{dyn}}} \frac{\partial p}{\partial \theta}, \quad S_\theta(t_0) = \frac{\partial p_{\text{init}}}{\partial \theta}. \quad (13)$$

The matrix $\partial p / \partial \theta$ is the *tangent* of the inputs, supplied through the `tangent` argument of `solveODE()` and fixed for the integration. The choice $\partial p / \partial \theta = I_{n_p}$ gives $S_\theta = S_p$; any other choice propagates M directions of θ -space at the cost of M additional state vectors.

Error norm with sensitivities

With forward sensitivities the integrator carries $S_\theta \in \mathbb{R}^{n_x \times M}$ beside the state, and every step produces $1 + M$ error vectors of length n_x : the state estimate $e_n^{(x)}$ and one estimate $e_S^{(k)}$ for each column $\partial x / \partial \theta_k$. `cppDE` combines them by the maximum of the per-vector WRMS norms, as `CVODES` does, rather than by a flat WRMS over all $n_x(1 + M)$ components.

The state norm $\|e_n^{(x)}\|_{\text{WRMS}}$ is exactly (10). The sensitivity column norm is its analogue, with S_θ replacing x in both the error vector and the weight,

$$\|e_S^{(k)}\|_{\text{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{i=1}^{n_x} \left(\frac{(e_S)_{ik}}{w_i^{(k)}} \right)^2}, \quad w_i^{(k)} = \text{atol} + \text{rtol} |(S_\theta)_{ik}^{\text{ref}}|, \quad (14)$$

in which $(S_\theta)_{ik}^{\text{ref}}$ is taken from the same source as x_n^{ref} : the accepted value for the multistep methods, the maximum over both step ends for the single-step methods. The step-size controller uses the largest of the state norm and the M column norms,

$$\|e_n\|_{\text{WRMS}} = \max \left\{ \|e_n^{(x)}\|_{\text{WRMS}}, \max_{1 \leq k \leq M} \|e_S^{(k)}\|_{\text{WRMS}} \right\}. \quad (15)$$

A flat WRMS divides by $n_x(1 + M)$ and averages the squared errors over the directions, so a large error in one column can be hidden by small errors in the others. The maximum holds every direction to the requested tolerance separately.

The corrector of BDF/NDF measures its corrections with the same maximum, so a sensitivity direction that has not converged causes further iterations or a smaller step even when the state has. With `solveODE(..., sensErrCon = FALSE)` the error test, the order selection, the corrector test and the initial-step estimates use the state components only, and the step sequence is that of a value run.

3.8 Events and restarts

cppDE supports two kinds of events that interrupt smooth integration. A *time-triggered event* fires at a user-specified time $t_e(\theta)$ that may depend on θ through a closed-form expression. A *root-triggered event* fires when a user-supplied event function $g_e(x, t)$ crosses zero along the trajectory; the firing time $t^*(\theta)$ is then defined implicitly by $g_e(x(t^*), t^*) = 0$. In both cases an event applies a discontinuous reset to the state,

$$x^+ = g(x^-, t),$$

for example the dosing of a kinetic species, a clipping rule that enforces non-negativity, or a switch between dynamical regimes.

Restart cost

Single-step methods (Rosenbrock4, Tsit5) resume after the reset with a full-order step. Multistep methods (BDF/NDF, Adams) restart at order 1 and rebuild the Nordsieck array over several steps, which dominates the cost on problems with frequent events.

For every method the step size is re-estimated from the post-event state, with the initial-step estimator of the solve and the remaining interval as upper bound. For a single-step method this is the whole restart.

Sensitivity transport across a discontinuity

The reset map g acts on the trajectory $x(t, \theta)$ at the firing time $t^*(\theta)$. The naive update of the sensitivity, by the linearisation of g around the pre-event state, $S_\theta^+ = (\partial g / \partial x) S_\theta^-$, is incorrect in the presence of an event because the firing time itself depends on θ . A perturbation δt^* of the firing time induces short integrations of $\dot{x} = f$ on either side of the event, with f taking different values before and after the reset. Denote by f^- the right-hand side just before the reset, evaluated at x^- , and by f^+ the right-hand side just after the reset, evaluated at $g(x^-, t^*)$. The sensitivity at the first grid time following the event then receives two contributions in addition to the linearised reset:

1. *Pre-event drift.* The pre-event trajectory x^- flows along $\dot{x} = f^-$ from the grid time $t_*^{(0)} := \text{scalar}(t^*)$ to the θ -perturbed firing time t^* , over a displacement $\delta t^* = t^* - t_*^{(0)}$ whose scalar part vanishes but whose θ -derivatives do not.
2. *Post-event drift.* The reset state $g(x^-, t^*)$ then flows along $\dot{x} = f^+$ from t^* back to the grid time over the negative of the same displacement.

Both shifts contribute terms of order $f^\pm (\partial t^* / \partial \theta)$ to the sensitivity at the grid time; to first order they give the *saltation matrix* of Section 3.8.

cppDE replaces the bare reset by a sequence of forward drift, reset and backward drift, evaluated in dual arithmetic. Each drift is one step of Heun's method, the explicit trapezoidal rule, which averages the

right-hand side at both ends of the interval. Since δt^* has scalar part zero, the value of x at the grid time is unchanged and only the θ -derivatives of the displacement enter the state. The reset g is applied between the drifts in the same arithmetic, so its first and second derivatives enter as well. Without sensitivities, g is applied as an ordinary map.

Time-triggered and root-triggered events differ in how δt enters: through the expression $t_e(\theta)$ in the first case, and through the implicit function theorem applied to $g_e = 0$ in the second.

Time-triggered events

When the firing time is given as an expression $t_e(\theta)$, its derivatives with respect to θ follow from the chain rule. The integrator advances on the scalar grid time $t_e^{(0)} := \text{scalar}(t_e)$ and treats the displacement

$$\delta t = t_e - t_e^{(0)}, \quad \text{scalar}(\delta t) = 0, \quad (16)$$

as a dual number whose value is zero but whose first-order tangents are $\partial t_e / \partial \theta_a$ and whose second-order tangents are $\partial^2 t_e / \partial \theta_a \partial \theta_b$. The state at the grid time $t_e^{(0)}$ is denoted x^- ; its first-order tangents are the pre-event sensitivities S_θ .

The transport consists of a Heun step along $\dot{x} = f$ from $t_e^{(0)}$ to t_e , the reset g at t_e , and a Heun step back from t_e to $t_e^{(0)}$ with the post-event right-hand side,

$$\begin{aligned} \text{forward: } f_1 &= f(x^-, t_e^{(0)}), \quad x_E = x^- + f_1 \delta t, \quad f_2 = f(x_E, t_e), \\ x^* &= x^- + \frac{1}{2}(f_1 + f_2) \delta t, \\ \text{reset: } x_+^* &= g(x^*, t_e), \\ \text{backward: } \tilde{f}_1 &= f(x_+^*, t_e), \quad x_B = x_+^* - \tilde{f}_1 \delta t, \quad \tilde{f}_2 = f(x_B, t_e^{(0)}), \\ x^+ &= x_+^* - \frac{1}{2}(\tilde{f}_1 + \tilde{f}_2) \delta t. \end{aligned} \quad (17)$$

An Euler step would suffice for first-order sensitivities: since $\text{scalar}(\delta t) = 0$, the powers δt^k for $k \geq 2$ have no first-order dual content, and any $O(\delta t^2)$ term drops out of $\partial x^+ / \partial \theta$. At second order the curvature term

$$\frac{1}{2}(Jf + \partial f / \partial t) \delta t^2, \quad J = \partial f / \partial x,$$

acquires a non-vanishing second-order dual component, because the square of δt has a non-zero mixed derivative

$$\frac{\partial^2 (\delta t)^2}{\partial \theta_a \partial \theta_b} = 2 \frac{\partial t_e}{\partial \theta_a} \frac{\partial t_e}{\partial \theta_b},$$

although its value and first derivatives are zero. Heun's second evaluation, at the shifted state and the shifted time t_e , supplies this term and leaves a truncation error of $O(\delta t^3)$. Evaluating f_2 and $\tilde{f}_2$ at $t_e^{(0)}$ instead would drop the $\frac{1}{2}(\partial f / \partial t) \delta t^2$ part for a right-hand side that depends explicitly on time.

Root-triggered events

For a root event the firing time $t^*(\theta)$ is defined implicitly by

$$G(t^*(\theta), \theta) := g_e(x(t^*(\theta), \theta), t^*(\theta)) = 0. \quad (18)$$

The event is handled in three steps: localisation of t^* on the scalar trajectory, reconstruction of the dual residual δt^* by the implicit function theorem, and the Heun transport (17) of the time-triggered case.

Localisation The integrator evaluates g_e at the end of every accepted step and at every requested output time. A crossing is accepted on $[t_n, t_{n+1}]$ when the two readings have opposite signs, and also when $g_e(x_{n+1}, t_{n+1})$ is exactly zero, as for a root on a sampled time. A zero at the lower end is not a new crossing, since it was reported with that sample; this also keeps an event from firing a second time on the surface it has just been applied on, where g_e is zero up to round-off.

The crossing is refined on the dense output of the accepted step, without further right-hand-side evaluations. The iteration is a false-position step whose interpolation factor is clamped to $[0.1, 0.9]$, which bounds the stagnation of the classical rule and keeps its speed on a nearly linear g_e . It stops when the bracket is narrower than the root tolerance, after at most fifty iterations, or when an iterate returns $g_e = 0$ exactly. It returns as $t_*^{(0)} := \text{scalar}(t^*)$ the end of the final bracket that still carries the sign g_e had before the crossing. The localisation says nothing about how t^* depends on θ , so the dual residual $\delta t^* = t^* - t_*^{(0)}$ is reconstructed from (18).

Dual residual of the event time If G is smooth at the crossing and $\partial G/\partial t \neq 0$ there, the implicit function theorem gives the derivative of $t^*(\theta)$ from $G(t^*, \theta) = 0$. Differentiating (18) once with respect to θ and applying the chain rule gives

$$\frac{\partial t^*}{\partial \theta} = -\frac{G_\theta}{G_t} = -\frac{\nabla_x g_e \cdot S_\theta}{\nabla_x g_e \cdot f + \partial g_e / \partial t} \Big|_*, \quad (19)$$

in which the denominator is the total time derivative $\dot{g}_e = \nabla_x g_e \cdot f + \partial g_e / \partial t$ along the trajectory at the crossing. The condition $\dot{g}_e \neq 0$ means that the trajectory crosses the event surface transversally; for a grazing crossing the firing time is not differentiable in θ , and the fallback described under simultaneous events applies. In dual-number form, (19) is realised as the quotient

$$\delta t^* = -\frac{g_e(x^-, t_*^{(0)})}{\dot{g}_e(x^-, t_*^{(0)})}, \quad (20)$$

where both g_e and $\dot{g}_e$ are evaluated on the dual-typed state x^- at the scalar grid time. The generated partials $\nabla_x g_e$ and $\partial g_e / \partial t$ are needed only for $\dot{g}_e$; the dual arithmetic forms $\nabla_x g_e \cdot S_\theta$ in the numerator. The scalar value of the numerator, of the order of the root tolerance, is set to zero before the division, so δt^* is zero in value and carries only derivatives.

The first-order part of (20) equals (19). At second order the quotient misses one term, because g_e is evaluated at the scalar time $t_*^{(0)}$ rather than at the dual time t^* , so the G_{tt} contribution from the time argument of g_e is absent. Expanding (18) to second order in θ and matching coefficients gives it. Writing $\delta t^* = \delta t_{(1)}^* + \delta t_{(2)}^* + \dots$ and using $G(t_*^{(0)}, \theta^{(0)}) = 0$,

$$0 = G_t \delta t_{(2)}^* + \frac{1}{2} G_{tt} (\delta t_{(1)}^*)^2 + G_{t\theta} \delta t_{(1)}^* \delta \theta + \frac{1}{2} G_{\theta\theta} \delta \theta^2,$$

of which the dual quotient supplies the $G_{t\theta}$ - and $G_{\theta\theta}$ -contributions but not the G_{tt} -contribution. The corrected update is therefore

$$\delta t^* \leftarrow \delta t^* - \frac{1}{2} \frac{G_{tt}}{G_t} (\delta t^*)^2, \quad (21)$$

where

$$G_{tt} = \frac{d^2}{dt^2} g_e(x(t), t) \Big|_*$$

is the second total time derivative of g_e along the trajectory. cppDE supplies G_{tt} from generated code or, where that is not available, by a forward finite difference of $\dot{g}_e$. Equation (21) changes only the second-order components of δt^* .

With δt^* known, the Heun transport (17) carries x^- across the event to the grid time $t_*^{(0)}$, with δt^* in place of δt and $t_*^{(0)} + \delta t^*$ in place of t_e .

Simultaneous events Two or more root conditions can cross zero within the same accepted step. cppDE localises each crossing separately and groups those that share $t_*^{(0)}$ within the localisation tolerance into a cluster. For a cluster the forward Heun shift is taken once to the common event surface, every triggered reset is applied there (the reset maps are assumed to commute), and one backward shift returns the state to

the grid time, at four right-hand-side evaluations whatever the number of events. The dual residual δt^* is computed once, from the first event in the cluster with non-zero $\dot{g}_e$, and shared by the others, which sit at the same grid time with the same parameter dependence through x^- .

If every event in the cluster has $\dot{g}_e = 0$ at the surface, the denominator of (19) vanishes and the implicit function theorem gives no $\partial t^*/\partial \theta$. cppDE detects this case by the size of $|\dot{g}_e|$ and applies the reset as a plain map without transport, which is exact in the limit where both contributions $f^\pm (\partial t^*/\partial \theta)$ vanish.

Conditions switched by a reset A reset is a jump, not a crossing of the continuous solution, so a root condition that a reset makes true escapes the sign-change search. cppDE therefore evaluates every condition on both sides of a jump. A condition whose sign changes across the jump fires there, one per sweep so that a reset can switch on the next condition; one that already held before the jump does not fire again.

The switched resets are applied on the surface of the jump, between the forward and the backward shift of the event that caused it, rather than after the transport has returned to the grid time. One Heun transport then carries the composition of both resets, and its sensitivities, to both orders, equal those of the same reset written as a time-triggered event at that time.

Terminal conditions are excluded from this path: they stop the integration rather than reset a state.

Connection to the saltation matrix

The classical treatment of sensitivity transport across a discontinuity is the *saltation matrix*, which adds to the reset Jacobian $\partial g/\partial x$ a term for the θ -dependence of the firing time:

$$S_\theta^+ = \frac{\partial g}{\partial x} S_\theta^- + \left(\frac{\partial g}{\partial t} + \frac{\partial g}{\partial x} f^- - f^+ \right) \frac{\partial t^*}{\partial \theta}.$$

The first-order part of the Heun transport equals this expression, for time-triggered events with $\partial t^*/\partial \theta = \partial t_e/\partial \theta$ and for root events with $\partial t^*/\partial \theta$ from (19). Unlike the saltation matrix, the transport also yields the second-order sensitivities. It resides in `cppde_saltation.hpp`, the event loop in `cppde_event_engine.hpp`.

3.9 Reverse-mode sensitivities: the discrete adjoint

Forward sensitivities cost one additional state vector per direction in θ , so a gradient of a scalar objective over M parameters costs M times the value alone. When the quantity of interest is a single scalar J , a likelihood or a residual sum of squares, the transpose of that computation delivers the whole gradient at a cost independent of M . cppDE builds this mode with `cppODE(..., derivMode = "reverse")`.

The reverse mode computes a contraction rather than the sensitivity matrix. Given a cotangent $W \in \mathbb{R}^{n_t \times n_x}$ attached to the observed trajectory, the reverse mode returns

$$\bar{\theta}_j = \sum_{o=1}^{n_t} \sum_{i=1}^{n_x} W_{oi} \frac{\partial x_i(t_o)}{\partial \theta_j}, \quad j = 1, \dots, n_x + n_p, \quad (22)$$

in one sweep. The row index o runs over the observation times and the column index i over the state components. Write $w_o \in \mathbb{R}^{n_x}$ for the vector formed from row o , the cotangent attached to the trajectory at t_o . For a least-squares objective W is the residual weighted by the error model, and θ is $\nabla_\theta J$ exactly. Several cotangents may be supplied as columns of a three-dimensional array, one sweep each.

The adjoint equation

The contraction (22) can be evaluated without forming S_p . Both backends implement the classical construction below, with different discretisations. Let S_p be the forward sensitivity of (12). Define $\lambda(t) \in \mathbb{R}^{n_x}$ backwards from the end of the observed interval by

$$\dot{\lambda} = -J(x, p, t)^\top \lambda, \quad \lambda(T^+) = 0, \quad \lambda(t_o^-) = \lambda(t_o^+) + w_o, \quad (23)$$

so that λ evolves continuously between observation times and jumps by the cotangent row at each one. Along the solution

$$\frac{d}{dt} (\lambda^\top S_p) = -\lambda^\top JS_p + \lambda^\top \left(JS_p + \frac{\partial f}{\partial p} \right) = \lambda^\top \frac{\partial f}{\partial p}, \quad (24)$$

where the two Jacobian terms cancel, so λ is defined without reference to θ . Integrating (24) over $[t_0, T]$ and collecting the jumps, whose contributions are exactly the terms of (22), gives

$$\bar{\theta} = \left(\frac{\partial x_0}{\partial \theta} \right)^\top \lambda(t_0) + \left(\frac{\partial p}{\partial \theta} \right)^\top \int_{t_0}^T \left(\frac{\partial f}{\partial p} \right)^\top \lambda(t) dt. \quad (25)$$

Taking θ to be the initial state and the parameters themselves makes both outer factors identities and splits the answer in two,

$$\bar{x}_0 = \lambda(t_0), \quad \bar{p} = \int_{t_0}^T \left(\frac{\partial f}{\partial p} \right)^\top \lambda(t) dt, \quad (26)$$

which is the pair a reverse solve reports. A reparametrisation $p = \Phi(\theta)$ is applied afterwards through the outer factors of (25), the chain rule of (13) applied on the left.

The gradient therefore costs one backward solve of a linear system of dimension n_x and a quadrature of dimension n_p , whatever M ; $\partial f / \partial p$ enters only through its contraction with λ and is never formed. The two parts of (26) are returned stacked as the rows of the **tangent** argument are indexed, states first and parameters second.

Discrete and continuous adjoints

Equation (23) describes the exact trajectory. Two discretisations of it are in common use, and `cppDE` provides both.

The *continuous* adjoint differentiates first and discretises afterwards: (23) is handed to an integrator of its own and solved backwards over forward states recovered by interpolation [6]. This is the route of `CVODES`, which `cnode()` exposes (Section 4.3).

The *discrete* adjoint reverses that order. It transposes the map the forward run applied, one accepted step at a time, and is what `cppODE()` implements. Both approximate the same gradient and converge as the tolerance is lowered. They differ in three respects.

Consistency of value and gradient. The discrete adjoint differentiates the trajectory that was actually computed, so it returns the derivative of the value the forward run reported, up to rounding. The continuous adjoint approximates the derivative of the exact solution, which no run computes, so its gradient and the accompanying value are consistent only to $O(\text{tol})$. Tightening the backward tolerance alone does not remove that difference.

Discontinuities. A jump in the state implies a jump in λ , given by the transposed saltation matrix of the event. A continuous adjoint has to restart its backward integration with that matrix at every event. In the discrete adjoint the jump is the transpose of arithmetic the forward run performed, handled like every other step.

Linear algebra per step. The adjoint of an implicit step is a single transposed solve against the iteration matrix of that step, evaluated at the state stored in its checkpoint, without a Newton iteration and without error control of its own. The continuous adjoint integrates a stiff linear ODE instead, with its own step-size control and step count.

The step map and its adjoint

Reverse accumulation has no counterpart in an augmented arithmetic. A dual number carries its derivative with its value, available as soon as an operation has executed, whereas an adjoint is a sum over all later uses of a quantity.

A step's adjoint can be obtained by recording its operations on a tape and traversing the record backwards, or by writing it in closed form, since the map is one accepted step of a known method on a fixed grid. `cppDE` uses the closed form and keeps no tape.

An accepted step of the multistep family decomposes into three parts:

$$z^{\text{pred}} = A z^{\text{in}}, \quad (27)$$

$$r(y, z^{\text{pred}}, \theta) = 0, \quad (28)$$

$$z^{\text{out}} = B z^{\text{pred}} + c a, \quad a = y - z_0^{\text{pred}}, \quad (29)$$

namely the Nordsieck rescale and Pascal shift, the corrector, and the tail that the controller applies between the acceptance of this step and that of the next. The operators A , B and c act on the slot index only; their coefficients follow from the carry and are identical for every state component. They are read off the stepper rather than transposed by hand: a probe whose states are the Nordsieck slots is driven through the routines the forward step runs, carrying the identity in the [slot $\times$ component] array, and one pass returns every column. The adjoint therefore uses exactly the coefficients of the forward step.

The corrector is differentiated by the implicit function theorem. The derivative $\partial r / \partial y$ is the iteration matrix $W = I - \gamma J$ that the forward step factorises, so one transposed solve against W replaces the entire Newton iteration. For the Rosenbrock family the corresponding object is a transposed stage recursion, six solves against one factorisation; for an explicit method it is the backward recursion over the tableau.

The contractions the adjoint requires are supplied by the model and emitted by the code generator beside the right-hand side: $J^T \lambda$, $(\partial f / \partial p)^T \lambda$ and, for a Rosenbrock method, the same two applied to Jv , which carries a second derivative because its stages solve against a matrix built from the Jacobian.

Checkpoint per step

The backward pass needs the states the forward run passed through. They are stored as one checkpoint per accepted step, holding the step-start state, the carry, the time, the step size, and a record of the operations the controller applied to the carry afterwards. The memory is linear in the number of steps and holds no derivatives.

The sweep re-forms and refactorises the iteration matrix at each checkpoint, since the factorisations of the forward run are not kept. The slot operators are cached, keyed on the carry and the step size, so the probe runs only where the controller changed the control state.

Observations lie at interpolated times rather than at step ends, and are evaluated through the continuous extension of the step that contains them. The adjoint of that evaluation is obtained from the same probe: after the tail has been applied the probe holds a valid interpolant, and evaluating it at the observation time yields the row $\partial x_{\text{interp}} / \partial (z^{\text{pred}}, a)$ directly.

Checkpoint reuse

The sweep uses only the checkpoints, so the forward pass and the backward pass can be issued as separate calls. A reverse solve with `keepStore = TRUE` returns its checkpoints as `$store`, and a later solve of the same model at the same times and parameters accepts them through `store`; that solve performs no integration and proceeds directly to the sweep. A store created at a different parameter value is rejected rather than reused. The cotangent may be omitted on the call that creates a store, which then performs a value-only run and retains its checkpoints.

A gradient requires two solves at the same parameter value, one to provide the values from which the cotangent is formed and one to perform the sweep, and the store removes the second integration. Several cotangents are handled in the same way: the columns of a three-dimensional cotangent sweep the one store in turn, so n_{seed} cotangents cost n_{seed} sweeps and a single integration.

Under `derivMode = "forward-reverse"` both arguments are refused. A checkpoint of that mode carries tangents, which live in the arena of the solve that recorded them and do not outlive it, so every second-order sweep is preceded by its own integration.

The adjoint of an event

An event is a map of its own between two steps. The forward run transports the discontinuity with the Heun sandwich of Section 3.8: a shift to the event surface, the resets, and a shift back to the grid time. Each shift is taken over the residual of the event time, which is zero in value and carries only derivatives. In value the jump is therefore a plain jump, and the sandwich reduces to the classical saltation relation,

$$dx = R'(dx_{\text{before}} + f_{\text{before}} s) - f_{\text{after}} s, \quad (30)$$

with R the resets, R' their Jacobian and s the differential of the event time. For a root event $g(x, t, p) = 0$ the implicit function theorem gives $s = -(\nabla g \cdot dx + \partial g / \partial p \cdot dp) / \dot{g}$, and for a fixed event at $t = \tau(p)$ it is $s = (\partial \tau / \partial p) \cdot dp$.

Since (30) is linear in dx_{before} and dp , its transpose can be read off directly. Contracting it with a cotangent λ_{after} and collecting the terms that multiply s leaves one scalar,

$$\kappa = \lambda_{\text{after}}^T (R' f_{\text{before}} - f_{\text{after}}), \quad \lambda_{\text{before}} = R'^T \lambda_{\text{after}} + \kappa \left(\frac{\partial s}{\partial x} \right)^T, \quad (31)$$

and the parameter cotangent receives $\kappa (\partial s / \partial p)^T$ in addition. For a root event both derivatives of s carry the factor $-\kappa / \dot{g}$, so the state picks up $-(\kappa / \dot{g}) \nabla g$ and the parameters $-(\kappa / \dot{g}) \partial g / \partial p$. For a fixed event $\partial s / \partial x = 0$ and the parameters pick up $\kappa \partial \tau / \partial p$.

The transpose R'^T is applied reset by reset and in reverse order. A reset that replaces a state removes the incoming cotangent of that component, one that adds to it leaves the cotangent unchanged, and one that multiplies it by h scales it by h ; where the reset value h depends on the state or on the parameters, its derivatives enter the same transpose. The adjoint of a jump therefore requires the right-hand side at the two ends, ∇g and $\dot{g}$, and the model's derivatives of the reset and event-time expressions. It requires no Jacobian of f , because every term the two shifts would contribute carries the residual as a factor and that factor is zero.

After a jump the multistepper restarts. `initialize` builds the Nordsieck history from a single state as $z_0 = x$ and $z_1 = h f(x, t)$, with the remaining slots zero, so the cotangent of the history collapses onto that one state. Writing w_j for the cotangent of slot j , the restart contributes

$$\bar{x} = w_0 + h J^T w_1, \quad \bar{p} = h \left(\frac{\partial f}{\partial p} \right)^T w_1, \quad (32)$$

and slots above the first contribute nothing. The same map applies at the start of the trajectory.

Second order of an event's adjoint

Write s for the differential of the event time, so that $s = 0$ in value and $ds \neq 0$. Then

$$s^2 = 0, \quad d(s^2) = 2s ds = 0, \quad d^2(s^2) = 2(ds)^2 \neq 0, \quad (33)$$

so terms of order s^2 drop from the gradient and not from the Hessian. Three of them enter a jump. The same three enter a fixed event, with the event time's own residual $\tau = \varepsilon(\theta) - \lfloor \varepsilon \rfloor$ in place of s : a fixed event is the same five-stage sandwich (39), and its transpose is that program reversed with τ substituted throughout. The classical saltation alone is exact in value and wrong in the tangents as soon as the jump height moves with a parameter, because $\bar{x}_a = (I - \tau J_a)^T \bar{x}_o$ then has a second factor that is not zero in value.

The first term is the curvature of the shift. Each half of the sandwich is a Heun step,

$$x_* = x_{\text{before}} + f_{\text{before}} s + \frac{1}{2} a_{\text{before}} s^2, \quad a = \frac{\partial f}{\partial x} f + \frac{\partial f}{\partial t}, \quad (34)$$

and a s is the difference of the two right-hand sides the step already evaluates, so no Jacobian is needed.

The second is the post-jump state, which the sweep reconstructs, since the store keeps x_{before} alone. Rebuilt from the bare reset, it would satisfy $\partial x_{\text{after}}/\partial t^* = 0$, whereas

$$\frac{\partial x_{\text{after}}}{\partial t^*} = f_{\text{before}} - f_{\text{after}}. \quad (35)$$

The reconstruction therefore runs the sandwich, not the reset.

The third comes from the quadrature. The parameter cotangent

$$\bar{p}_k = \int_0^T G_k dt = \int_0^{t^*} + \int_{t^*}^T, \quad G_k = \left(\frac{\partial f}{\partial p_k} \right)^\top \lambda, \quad (36)$$

is split at t^* , and G_k is discontinuous there because the state is. Differentiating a split integral whose split point moves leaves the jump of the integrand at the split,

$$\frac{\partial \bar{p}_k}{\partial v} \supset \left(G_k(t^{*-}) - G_k(t^{*+}) \right) \frac{\partial t^*}{\partial v}. \quad (37)$$

Only parameters whose $\partial f/\partial p_k$ depends on a state the reset changed contribute.

Implemented as accumulations scaled by the residual, all three are zero in value and carry $\partial t^*/\partial v$ in their tangent, so the gradient is unchanged.

For a fixed event at $t = \tau(p)$ the residual is $\tau - \lfloor \tau \rfloor$. For a root event it is

$$s = -\frac{g - \lfloor g \rfloor}{\dot{g}}, \quad \dot{g} = \frac{\partial g}{\partial t} + \nabla g \cdot f, \quad (38)$$

read at the stored state. Localisation ignores the tangents, so that state carries $\partial x/\partial \theta$ at a fixed time and ∇g contracts it to the event time's.

Transposing the root sandwich

The jump is the composition

$$x_e = x_b + f_b s, \quad x_* = x_b + \frac{1}{2}(f_b + f_e)s, \quad x_a = R(x_*), \quad x_k = x_a - f_a s, \quad x_o = x_a - \frac{1}{2}(f_a + f_k)s, \quad (39)$$

with $f_e = f(x_e, t^*)$, $f_a = f(x_a, t^*)$ and $f_k = f(x_k, t_0)$. Equation (39) is a straight-line program, and its adjoint is obtained by reversing it assignment by assignment rather than by collecting a closed form. A collected form pruned by order in s , keeping a product only where one factor does not vanish in value, is valid for the gradient and invalid for the Hessian: by (33), s^2 has vanishing value and vanishing first derivative, while $d^2(s^2) = 2(ds)^2 \neq 0$.

Reversed assignment by assignment, the shift receives a contribution from every assignment in which it occurs. Two of these arise because f_e and f_a are evaluated at t^* , which depends on θ :

$$\bar{s} \supset \left(\frac{\partial f}{\partial t} \right)_{(x_a, t^*)} \cdot \bar{f}_a + \left(\frac{\partial f}{\partial t} \right)_{(x_e, t^*)} \cdot \bar{f}_e. \quad (40)$$

Both cotangents carry a factor s , so (40) is absent from the gradient and contributes at full strength to the Hessian. It vanishes identically for an autonomous right-hand side, and a pruned form omits it whenever $\partial f/\partial t \neq 0$.

The forward path corrects the shift by $-\frac{1}{2}(\ddot{g}/\dot{g})s^2$, whose chain factor $1 + 2c_2 s$, $c_2 = -\ddot{g}/(2\dot{g})$, is one in value and a second-order tangent; the transpose carries it on $\partial s/\partial x$.

The implementation is checked in three ways. With $R = \text{id}$ the composition (39) is the identity and the transpose cancels term by term. The transpose is compared with a nested-dual evaluation of the same

sandwich, without a solver, a store or an event engine. And a whole trajectory is compared with forward over forward on one recorded step sequence. Only the third is sensitive to (40): the second is checked against the same forward routine the transpose reverses, so a term absent from both sides cancels.

The shift has a derivative of its own. Since $\dot{g}$ reads the state,

$$\frac{\partial s}{\partial x} = -\frac{\nabla g}{\dot{g}} - \frac{s}{\dot{g}} \nabla \dot{g}, \quad \nabla \dot{g} = \nabla \frac{\partial g}{\partial t} + (\nabla^2 g) f + J^\top \nabla g, \quad (41)$$

and the second term carries g , which is zero in value: absent from the gradient and full strength in the Hessian. For a g that is linear in the state and does not depend on time, $\nabla \dot{g}$ is $J^\top \nabla g$ alone, so that case does not test the other two terms. The generator therefore differentiates $\dot{g} = \partial g / \partial t + \nabla g \cdot f$ as one expression rather than assembling three terms, which gives $\nabla \partial g / \partial t$ and $(\nabla^2 g) f$ with it, and on θ the matching $\partial^2 g / \partial \theta \partial t$, $(\partial \nabla g / \partial \theta) \cdot f$ and $(\partial f / \partial \theta)^\top \nabla g$. A forcing inside g is differentiated in t through its rate: the time derivative on the expression graph seeds each forcing leaf with a rate leaf, which prints as the derivative of the PCHIP interpolant.

Time-dependent resets. A reset value h that depends explicitly on time is applied on the surface, at t^* , so

$$\frac{\partial x_a}{\partial t^*} \supset c \frac{\partial h}{\partial t}, \quad c = \begin{cases} 1 & \text{add} \\ 1 & \text{replace} \\ x_{*k} & \text{multiply,} \end{cases} \quad (42)$$

which contributes to κ , the scalar multiplying the shift. This is a first-order term: omitting it makes the gradient wrong, not only the Hessian. For the same reason the transposed reset receives the event time as a dual rather than as a plain number, so that $\partial h / \partial \theta$ carries the dependence of t^* on θ .

Such a value inherits the root tolerance rather than the integration tolerance, since it evaluates t^* directly, whereas in a value that depends only on the state at t^* the localisation error is damped by the smoothness of the trajectory.

Against the sandwich itself, evaluated over a nested dual with no solver around it, the transpose is exact to $9 \cdot 10^{-16}$, and over a whole trajectory the second order agrees with the forward mode to the tolerance of the solve.

Every buffer the sweep hands to a generated contraction has its tangent storage bound by the sweep. The contraction opens an arena scope of its own, and an output without bound storage would keep a tangent pointer into memory that scope reclaims on exit. Since every read of such a buffer in the jump transpose is multiplied by s , the value and the gradient would be unaffected, and the Hessian would be wrong by a rank-one term.

Verifying a derivative across a moving event

When t^* depends on a parameter, the reported trajectory carries a row whose time moves with it. A functional $w^\top x$ seeded on that row is not the same functional at two parameter values and has no derivative to compare against. Restricting the cotangent to the user's own output times restores a well-posed functional, after which the forward, forward-over-forward and difference-quotient answers agree to the tolerance of the solve.

Internal numerical differentiation

The step sizes the controller selected are constants of the map that is differentiated. This is Bock's internal numerical differentiation: the nominal run adapts freely, and the derivative is taken of the scheme that run applied. Differentiating the control law instead introduces derivatives of the step sizes into the chain rule, which makes the discrete adjoint inconsistent with the adjoint ODE. That term, measured at $O(\text{tol})$, is left out.

This choice is often justified by the claim that h is piecewise constant in θ , with zero derivative almost everywhere. An adaptive controller does not produce such an h , and the argument below does not rely on it.

Let $y_h(\theta)$ denote the result the program returns on the grid the controller selects at θ . Then

$$\frac{dy_h}{d\theta} = \frac{\partial y_h}{\partial \theta} + \sum_k \frac{\partial y_h}{\partial h_k} \frac{dh_k}{d\theta}, \quad (43)$$

where the first term is what internal numerical differentiation computes and the sum is the grid effect, which differentiating the control law would add.

The function $h(\theta)$ is piecewise smooth with jumps. Within an acceptance region it varies smoothly through the error estimate; at the boundary of one a step that was accepted is rejected and the entire later grid shifts. Halving the spacing of a sweep in θ doubles the measured magnitude of the grid effect, which identifies it as a jump height divided by $\Delta\theta$ rather than as a derivative. Automatic differentiation reproduces only the smooth part. Differentiating the control law therefore returns the exact gradient plus the smooth part of the grid effect, with the jump part, which is the larger of the two, absent. The result is neither the derivative of the exact solution, to which the addition does not belong, nor the derivative of the reported value, of which the dominant contribution is missing. Recovering the grid effect requires a smoothed controller, a fixed grid or a randomised estimator, none of which differentiation of the existing chain provides.

By the same criterion it is the forward mode that departs from internal numerical differentiation, not the reverse one. The value pass of the reverse mode runs the stepper on plain numbers, so the step sizes follow from the state error alone and the derivative is taken on the grid of the nominal run. The forward mode with the default `sensErrCon = TRUE` maximises its error norm over the sensitivity columns as in (15) and therefore integrates on a finer grid, to which no reported value belongs. The reverse gradient is the derivative of the trajectory a value run returns, the forward gradient the derivative of a trajectory that is not reported, and the $O(\text{tol})$ difference between them is mainly due to the forward mode.

The forward mode keeps the sensitivities in its error norm by default: they are integrated alongside the state and their errors accumulate, which CVODES guards against in the same way (`CVodeSetSensErrCon`). Value and gradient on slightly different grids is the smaller cost. `solveODE(..., sensErrCon = FALSE)` selects the alternative.

Goal-oriented step-size control

A grid selected from the state error alone need not be fine enough for the gradient. Since λ is solved backwards, a step at which λ is large contributes strongly to the error in J even where its own state error is small; the gradient inherits the order of the state error but not its constant. In the forward mode this is settled within the run, by maximising the error norm over the sensitivity columns as in (15). In the reverse mode it cannot be, because λ at step k depends on every step after it and is not available while step k is being taken.

It is available across runs. The `errWeights` argument of `solveODE()` takes the λ left by an earlier sweep, sampled on that sweep's grid, reads it back by linear interpolation and enters it into the error norm as

$$\text{err} = \max \left(\text{err}_{\text{state}}, \frac{|\lambda(t_{n+1})^\top e_n|}{\text{gradtol}} \right), \quad (44)$$

where e_n is the embedded error estimate of the step. The numerator estimates the contribution of this step to the error in the objective, and `gradtol` is the tolerance to which that contribution is held, which makes the two terms comparable.

Because of the maximum, the grid can only become finer than `abstol` and `reltol` require, never coarser, so the accuracy of the trajectory is unaffected. An inaccurate λ costs at most computing time: if it is too large the steps are unnecessarily small, and if it is too small the state term governs as before. An estimate of λ from a neighbouring parameter value is therefore admissible, so an optimiser can pass the sweep of iteration $i - 1$ to iteration i and pass for one pass rather than two.

The weight is an input to the controller and is held in plain `double`. The controller is not differentiated, and a weight carried into the derivative would reintroduce the step-size term discussed above. The interpolation is linear rather than PCHIP for the same reason and for two further ones: λ jumps where the objective seeds

it and where an event resets the state, so **breaks** names the sample indices an interpolant must not span, and a query inside such an interval takes the nearer endpoint; and shape preservation is of no value for a weight whose overshoots cost only time. A **floor** raises weights that fall below a given fraction of the largest, which is a precaution rather than a necessity, since λ measures a linearised influence and a state it weights near zero may still move nonlinearly.

The indicator can be inspected directly. With `adjointGrid = TRUE` a reverse solve also returns the grid the sweep ran on, λ at the start of each step, and $\eta_n = \lambda(t_{n+1})^\top e_n$ for each step, which is the numerator of (44) and estimates that step's contribution to the error in J . The estimate is of first order, since it evaluates λ at the computed rather than the exact state, and what the adaptation requires of it is the ranking of the steps rather than the magnitude.

Relation to the forward mode

Forward and reverse derivatives of the same model do not agree to machine precision. With `sensErrCon = TRUE` a sensitivity run maximises its error norm over the state and every tangent column and takes a finer grid than a value run at equal tolerances. The two modes therefore differentiate two different discretisations, each exactly, and the difference falls with the tolerance.

The reverse gradient is the derivative of the trajectory a value run produces, so the value and the gradient returned to the caller are consistent. A trust-region method, which compares a predicted with an achieved reduction, relies on this.

Of the four methods, the Rosenbrock adjoint is the most accurate: its stages are direct solves, so its adjoint is the exact transpose of the discrete step, whereas the adjoint of a corrector applies the implicit function theorem to an iteration stopped at a finite tolerance.

The reverse mode supports all four time-stepping methods, time- and root-triggered events, forcings, the batch entry point and the sparse linear solver. The implementation resides in `cppde_adjoint_step.hpp`, `cppde_reverse_step.hpp` and `cppde_reverse_trajectory.hpp`.

Second derivatives: forward over reverse

For a fixed cotangent, (22) defines the gradient map

$$\bar{\theta} : \mathbb{R}^M \rightarrow \mathbb{R}^M, \quad \theta \mapsto \nabla_\theta J(\theta), \quad (45)$$

whose Jacobian is the Hessian $\nabla_\theta^2 J$. Nothing in the sweep that evaluates (45) depends on the scalar type, so instantiating it on `dual<double, N>` rather than on `double` differentiates the sweep itself. Seeded with the tangent $V \in \mathbb{R}^{M \times N}$ it returns the pair $(\nabla_\theta J, \nabla_\theta^2 J V)$, both computed exactly rather than by a difference quotient; for $V = I_M$ the second member is the Hessian. This is selected by `derivMode = "forward-reverse"` and reported as `curvature`, of shape $[n_x + n_p, N, n_{\text{seed}}]$, beside the `cotangent` and `tangent` of the same run.

The tangents satisfy the second-order adjoint system. Write $v = (v_0, v_p)$ for one direction in θ , let σ be the directional sensitivity and μ the tangent of λ along v . Then σ solves (12) contracted with v ,

$$\dot{\sigma} = \frac{\partial f}{\partial x} \sigma + \frac{\partial f}{\partial p} v_p, \quad \sigma(t_0) = v_0, \quad (46)$$

and differentiating (23) along v gives μ backwards from $\mu(T^+) = 0$,

$$\dot{\mu}_i = - \sum_k \frac{\partial f_k}{\partial x_i} \mu_k - \sum_k \lambda_k \left(\sum_j \frac{\partial^2 f_k}{\partial x_i \partial x_j} \sigma_j + \sum_m \frac{\partial^2 f_k}{\partial x_i \partial p_m} v_m \right), \quad (47)$$

and with the jumps $\mu(t_o^-) = \mu(t_o^+) + \dot{w}_o$, where $\dot{w}_o$ is the derivative of the cotangent row w_o along v . It is zero for a cotangent that does not depend on θ ; a cotangent formed from the solution, such as a weighted residual, supplies it through the `curvature` argument, an array $[n_{\text{out}}, n_x, n_{\text{seed}}, N]$. Without it the gradient is

unchanged and the Hessian misses the dependence of the cotangent on θ . Differentiating (26) along v then gives the two halves of $\nabla_{\theta}^2 J v$, the state half as $\mu(t_0)$ and the parameter half as

$$\int_{t_0}^T \left(\sum_k \frac{\partial f_k}{\partial p_m} \mu_k + \sum_k \lambda_k \left(\sum_j \frac{\partial^2 f_k}{\partial p_m \partial x_j} \sigma_j + \sum_l \frac{\partial^2 f_k}{\partial p_m \partial p_l} v_l \right) \right) dt. \quad (48)$$

cppDE does not integrate (46) to (48). It obtains their discrete counterpart from the dual instantiation: the tangent carried alongside λ is μ on the grid the forward run chose, and the tangent of the parameter accumulator is (48). The second derivatives of f that appear in (47) and (48) are supplied by instantiating the first-order contractions $J^T \lambda$ and $(\partial f / \partial p)^T \lambda$ over duals, so forward over reverse needs no generated expression beyond those of the first-order sweep. For a Rosenbrock method these already include the contractions of Jv , which are second derivatives of f .

The tangents have to be present in the forward pass as well, since the sweep reads its checkpoints, which makes that pass a sensitivity run in the same N directions. Its error control reads the values only, so it takes the grid of a value run. The cost is one sensitivity integration and one sweep, both of width N , and therefore linear in N . Forward over forward, selected by `derivMode = "forward-forward"` or by the older `deriv2 = TRUE`, propagates the full second-order sensitivity tensor through the integration at a cost quadratic in M . A difference quotient of the gradient would cost N further gradient evaluations and lose half of the available digits.

Forward over forward answers a different question. It returns `hessian`, the second derivatives of every state with respect to every pair of parameters, from which the Hessian of any functional can be contracted afterwards. Forward over reverse returns the Hessian of one functional. Where that functional is a likelihood the mixed route is the cheaper by a factor of M ; where the states themselves are of interest, only `hessian` answers.

The second order needs two further facilities. The iteration matrix carries tangents, so the transposed solve applies the implicit function theorem with the matrix transposed alongside; `cppde_ad_lu.hpp` and `cppde_sparse_ad_lu.hpp` provide this in the dense and the sparse case. And a generated model body opens an arena scope of its own, in which a tangent allocated for one of the caller's numbers would be released, so the sweep binds the tangent storage of every buffer before passing it to the model, as the stepper does for its own buffers through the tangent slab.

The two second-order routes agree to 10^{-6} on `bdf`, `rb4` and `tsit5`, and all four methods return the same Hessian in reverse to 10^{-5} . The second comparison is the stronger one: the four methods adapt grids of their own, so agreement means that each sweep differentiated the discretisation its own method applied.

3.10 Batches of conditions

`solveODEBatch()` solves K conditions of one model in a single `.Call`, in three phases. The first reads the arguments of every condition from R into plain C++ structures, on the main thread. The second runs the K solves on OpenMP threads with dynamic scheduling, so a slow condition does not hold up the others; the workers touch neither the R API nor R's allocator, which is not thread-safe. The third builds the result list, again on the main thread. When the output grid is fixed by `times`, the result arrays are allocated before the second phase and the workers write into them directly; a root event leaves the row count open until the solve ends, and its output is copied afterwards.

Each condition is solved as `solveODE()` would solve it: the workers share no state and form no reductions across conditions, so the result is bitwise identical to the serial loop, whatever the thread count and schedule.

The thread count is the smaller of the request and K . It falls to one inside an enclosing OpenMP region, where the caller already runs in parallel, and in a child created by `fork()`, because the OpenMP thread pool does not survive a fork; a `pthread_atfork` handler marks the child. The BLAS library's own threads are set to one for the duration of a batch and restored afterwards, and the same handler does so around every `fork()`. `forkGuard()` reports which BLAS library was found, and `batchAvailable()` whether a model runs its conditions in parallel.

`prepareBatch()` performs the first phase once and keeps the converted arguments in a handle. `solveBatch()` then solves the same conditions with new parameter values, tangents, cotangents or error weights of unchanged shapes, as an optimiser does at every iteration. Reverse-mode solves run through the same entry with one cotangent per condition, and `cvode()` models provide the same batch entry.

4 `cvode()`

The `cvode()` entry point compiles a model against the system-installed SUNDIALS CVODE/CVODES library and exposes the result through the same `solveODE()` interface as `cppODE()`. The code generator, the `events`, `forcings`, `rootfunc` and `fixed` arguments, the shape of the returned object, and the column ordering of `parms` and `tangent` are the same for both backends. SUNDIALS (≥ 6.0) must be installed on the build host; KLU, used for sparse Jacobians, is detected the same way and is required when `sparse = TRUE`.

4.1 What is shared

The Jacobian $J = \partial f / \partial x$ is formed on the expression graph of Section 2 and emitted as C++ as in `cppODE()`, including its linear tables and loops. Both backends select the sparse KLU solver automatically for $n_x \geq 8$ states and a structural density of J of at most 0.4, provided SuiteSparse was found at install time, and dense LAPACK otherwise. Forcing functions are PCHIP interpolants, events apply a discontinuous reset to the state, and root events fire when a user-supplied scalar function crosses zero.

4.2 What differs

Sensitivity right-hand side. CVODES integrates the forward-sensitivity equation

$$\dot{S}_k = J(x, p) S_k + \frac{\partial f}{\partial p_k}, \quad k = 1, \dots, n_p, \quad (49)$$

directly. The generated right-hand side of (49) is one Jacobian-vector product of f in the direction $(S_k, \partial p / \partial \theta_k)$, evaluated in `double` without forming J . The in-tree backend recovers the same equation by evaluating f on dual-typed state, cf. (12). The two formulations are mathematically equivalent. Numerically, the generated product avoids the per-evaluation arena allocation required by the dual-number path, but is invoked once per direction, whereas the dual evaluation carries all directions in a single pass.

Methods. CVODE provides only the multistep families, selected by `cvode(method = "bdf")` or `cvode(method = "adams")`; `rb4` and `tsit5` are not available. The stability properties of Section 3 apply unchanged.

Second-order sensitivities. CVODES provides only first-order forward sensitivities, and `cvode()` does not accept `deriv2 = TRUE`.

Events. Events on `cvode()` models use the Heun transport of `cppODE()`, with the first-order corrections (17) and (19). The transport runs in the generated glue rather than inside CVODES, which re-initialises its sensitivities from the transported state.

Seeding. The chain-rule construction $S_\theta = S_p \partial p / \partial \theta$ of equation (13) is identical to the `cppODE()` case. `solveODE()` accepts the same `tangent` matrix shapes (state-only, full, and partial) and the same row-name conventions; only second-order S_θ is unavailable.

4.3 Adjoint sensitivity analysis

`cvode(..., derivMode = "reverse")` compiles the CVODES adjoint sensitivity analysis (ASA) in place of the forward sensitivities and returns the same `$cotangent` as a native reverse solve, the contraction (22) of a cotangent, stacked states first and parameters second. The rest of the call, including the batch entry point, is unchanged.

Backward problem. Where `cppODE()` transposes the steps the forward run took, CVODES integrates (23) as an ODE in its own right [6]. The generated glue emits the two transposed contractions as vector-Jacobian products of f (Section 2),

$$\dot{\lambda} = - \left(\frac{\partial f}{\partial x} \right)^\top \lambda, \quad \dot{q} = - \left(\frac{\partial f}{\partial p} \right)^\top \lambda, \quad (50)$$

the second of which is the quadrature of (26) with $q(T) = 0$. CVODES carries it alongside the backward state and leaves it out of the error test, its default: with gradient entries of very different scale the test would stall the backward step. Both equations are integrated from T down to t_0 , and the minus signs of (50) cancel against the reversed limits, so $q(t_0)$ is the parameter part of the gradient and $\lambda(t_0)$ the state part. Both right-hand sides are the contractions the native reverse mode uses; neither forms J .

Checkpoints. The backward right-hand side needs $x(t)$, which the forward pass no longer holds. CVODES stores a checkpoint every `asaCheckpoints` accepted steps, 200 by default, and recovers the states between two checkpoints by re-integrating forward from the earlier one and interpolating; the argument trades memory against that re-integration. This is the main difference between the two discretisations: the adjoint sees interpolated states rather than the ones the forward run stepped through, so the gradient is the derivative of no discretisation that was carried out. It approximates the exact gradient to $O(\text{tol})$ and differs by the same order from the derivative of the value the call reported.

Jumps at observations. λ is initialised with the cotangent row of the last observation, and the sweep stops at every earlier observation time to add that row, the jump condition in (23). A jump is a new initial condition, so the backward problem is re-initialised wherever a row is nonzero, and only there: re-initialising at every output time would reduce the method to first order at times the objective does not observe. The backward problem is a BDF solve with the linear solver of the forward problem, KLU for a sparse model, and the analytic Jacobian $-J^\top$, formed from J in the forward pattern; it runs at the caller's tolerances and step budget.

Several cotangents. The columns of a cotangent share the checkpoints, but each needs a backward integration of its own. The native mode sweeps its stored checkpoints once per column instead. Its `keepStore`, `store`, `adjointGrid` and `errWeights` arguments have no counterpart here, since CVODES holds its own checkpoints and runs its own backward controller.

Restrictions. Events and root functions are rejected at code-generation time, since a jump in the state implies a jump in λ and ASA has no interface to apply one. Forward sensitivities cannot be requested for the same model, and there is no second-order mode. An event model, a Hessian, or a gradient consistent with the value it accompanies therefore requires the native backend.

ASA is an independent implementation of the same mathematics and serves as a check on the in-tree adjoint: the two agree to 10^{-6} on the test models.

5 cppFUN()

`cppFUN()` compiles a system of algebraic expressions

$$y_i = g_i(x_1, \dots, x_{n_x}, p_1, \dots, p_{n_p}), \quad i = 1, \dots, n_y, \quad (51)$$

with its first and second derivatives. The variables x are supplied per observation, as the rows of an $n_{\text{obs}} \times n_x$ matrix, and the parameters p are shared by all rows. There is no time integration. Typical uses are observation maps, which connect integrator output to data and whose derivatives enter the gradient of a likelihood, and reparametrisations $p = \Phi(\theta)$, whose Jacobian is the tangent of (13).

The expressions are parsed into the expression graph of Section 2, and the generator emits one function template over the scalar type that evaluates g for one observation. It is instantiated on `double`, on `cppde::dual<double, 0>` and on the nested dual `cppde::dual2nd<double, 0>`. The vector-Jacobian product is a reverse sweep on the same graph, emitted as a second template and instantiated on `double` and on the dual.

Every entry runs compiled code. With the default `compile = FALSE` the constructor only writes the source, and the returned object has to be passed to `compile()` before use; calling a function of an uncompiled object stops with an error.

5.1 Entry points and derivative directions

The argument `derivMode` names the derivative directions that are built, `"forward"`, `"reverse"` or both, which is the default; naming one direction omits the entries of the other, and their compile time. `deriv` and `deriv2` switch the first- and second-order entries, and `deriv2 = TRUE` requires `"forward"`. Table 4 lists the R functions of the returned object and the compiled entries they call.

R function	direction	compiled entry
<code>func(x, p)</code>	both	<code>_eval_c</code>
<code>jac(x, p[, tangentX, tangentP])</code>	forward	<code>_eval_ad_c</code>
<code>hess(x, p[, tangentX, tangentP, hessianX, hessianP])</code>	forward	<code>_eval_ad2_c</code>
<code>evaluate(..., deriv2)</code>	forward	<code>_eval_ad_c</code> or <code>_eval_ad2_c</code>
<code>evaluateBatch(sets, cores, deriv2)</code>	forward	<code>_eval_ad_batch</code> or <code>_eval_ad2_batch</code>
<code>vjp(x, p, cotangent)</code>	reverse	<code>_vjp_c</code>
<code>vjp(x, p, cotangent, tangentX, tangentP, curvature)</code>	reverse	<code>_vjp_ad_c</code>

Table 4: R functions of a `cppFUN()` object and the compiled entries they call, each prefixed by the model name. The `_c` entries are `.Call` interfaces that take their arguments by reference; `_eval`, `_eval_ad`, `_eval_ad2` and `_vjp` also exist as `.C` interfaces, used only for a shared object without the `.Call` symbol.

Forward direction. `jac` evaluates g on duals and returns $\partial y / \partial \theta$ as an array $[n_{\text{obs}}, n_y, n_\theta]$, `hess` evaluates it on nested duals and returns $\partial^2 y / \partial \theta^2$ as $[n_{\text{obs}}, n_y, n_\theta, n_\theta]$. The directions θ are given by the tangents of the next section. Without them, the identity over the variables and the differentiated parameters is used, so the result is the Jacobian or Hessian with respect to (x, p) . `evaluate` returns the value and the derivatives of one pass, which separate calls of `func`, `jac` and `hess` would evaluate up to three times, as `y`, `tangent` and, with `deriv2 = TRUE`, `hessian`. `evaluateBatch` runs `evaluate` over a list of argument sets in one call, distributed over OpenMP threads with dynamic scheduling, and falls back to a loop for fewer than two sets.

Reverse direction. For a cotangent w of the outputs, $[n_{\text{obs}}, n_y]$ or $[n_{\text{obs}}, n_y, n_{\text{seed}}]$, `vjp` returns

$$(w_x)_{ojs} = \sum_i w_{ois} \frac{\partial g_i}{\partial x_j}(x_o, p), \quad (w_p)_{ks} = \sum_o \sum_i w_{ois} \frac{\partial g_i}{\partial p_k}(x_o, p), \quad (52)$$

as `cotangentX` and `cotangentP`, together with the value y . The parameter `cotangent` is summed over the observations because the parameters are shared by them. The contraction is one reverse sweep per observation and cotangent column, at a cost independent of the number of upstream parameters, and forms neither the Jacobian nor the Hessian. Statements of the sweep that do not depend on the cotangent are evaluated once per observation, outside the loop over cotangent columns.

Given `tangentX` and `tangentP`, the tangents v_x and v_p of the inputs in n_{dir} directions, or `curvature`, the derivative $\dot{w}$ of the cotangent along them, `vjp` runs the same template on duals and returns, beside `cotangentX` and `cotangentP`, the derivatives

$$\frac{d}{dv}(w^\top J) = w^\top \frac{\partial J}{\partial(x, p)} v + \dot{w}^\top J, \quad J = \frac{\partial g}{\partial(x, p)}, \quad (53)$$

as `curvatureX` and `curvatureP`. This is forward over reverse as in Section 3.9: an observation map contributes its curvature to a backward chain without forming a Hessian.

5.2 Tangent, hessian, cotangent and curvature

A map has a tangent and a Hessian, a functional has a cotangent and a curvature. For the upstream map $\Phi : \theta \mapsto (x, p)$ these are the `tangent` $\partial \Phi(\theta)$ and the `hessian` $\partial^2 \Phi(\theta)$. For a scalar functional L of the outputs

y they are the **cotangent** $\partial L(y)$ and the **curvature** $\partial^2 L(y) \partial y$, the derivative of the cotangent along the tangent. An argument and its result carry the same name: a forward argument lives on the inputs and its result on the outputs, a reverse argument lives on the outputs and its result on the inputs. Table 5 lists the names.

<code>solveODE()</code>	<code>cppFUN()</code> argument	<code>cppFUN()</code> result
tangent	tangentX , tangentP	tangent
hessian	hessianX , hessianP	hessian
cotangent	cotangent	cotangentX , cotangentP
curvature	curvature	curvatureX , curvatureP

Table 5: Names of the four derivative objects in R. `solveODE()` uses one name for the argument and the element of its result; the `cppFUN()` results are elements of the lists that `evaluate` and `vjp` return.

Forward over forward takes the **hessian** of the inputs and returns the **hessian** of the outputs, the second derivatives of every output in every pair of directions of the tangent. Forward over reverse takes the **curvature** of the outputs and returns that of the inputs, the Hessian of the seeded functional applied to the tangent; with the identity tangent, the columns of that Hessian.

Chain rule. `cppFUN()` takes the tangent and the hessian of its inputs in two blocks each:

- **tangentX[obs, name, theta]**: $\partial x_i / \partial \theta_a$, per observation row, variable and direction θ_a ;
- **tangentP[name, theta]**: $\partial p_j / \partial \theta_a$, the same for all observations;
- **hessianX[obs, name, theta1, theta2]**: $\partial^2 x_i / \partial \theta_a \partial \theta_b$, optional, zero by default;
- **hessianP[name, theta1, theta2]**: $\partial^2 p_j / \partial \theta_a \partial \theta_b$, optional, zero by default.

They may also be supplied as the attributes **deriv** and **deriv2** of the variables and parameters. Tangent rows of parameters listed in **fixed** are set to zero, and symbols absent from the tangent contribute zero.

With the tangent $\partial \Phi \in \mathbb{R}^{n_{\text{obs}} \times n_{\text{diff}} \times n_{\theta}}$ over the $n_{\text{diff}} = n_x + n_p$ inputs and the hessian $\partial^2 \Phi$, the chain rule for the Jacobian reads

$$\left. \frac{\partial y}{\partial \theta} \right|_{\text{obs}} = J(x_{\text{obs}}, p) \cdot \partial \Phi_{\text{obs}}, \quad (54)$$

and for the Hessian of output o two contributions combine,

$$\left. \frac{\partial^2 y_o}{\partial \theta \partial \theta^\top} \right|_{\text{obs}} = \partial \Phi_{\text{obs}}^\top H_o(x_{\text{obs}}, p) \partial \Phi_{\text{obs}} + \sum_i J_{o,i}(x_{\text{obs}}, p) \partial^2 \Phi_{\text{obs}, i, \cdot, \cdot}, \quad (55)$$

the second of which vanishes when Φ is affine.

Neither J nor H is formed. Both contractions follow from seeding the tangent slots of the duals. Each input scalar x_i becomes

$$\tilde{x}_i = (x_i + \partial \Phi_{i, \cdot} \varepsilon_{\text{in}}) + (\partial \Phi_{i, \cdot} + \partial^2 \Phi_{i, \cdot, \cdot} \varepsilon_{\text{in}}) \varepsilon_{\text{out}},$$

a nested dual whose inner tangent slots carry $\partial x_i / \partial \theta$ and whose outer tangent slot k is itself an inner dual with value $\partial x_i / \partial \theta_k$ and inner tangents $\partial^2 x_i / \partial \theta_k \partial \theta$. The multiplication rule then propagates both the $\partial \Phi^\top H \partial \Phi$ and the $J \cdot \partial^2 \Phi$ terms in one evaluation; the inner tangents of $\tilde{y}$ are $\partial y / \partial \theta$, and the cross slots $\tilde{y}[k][m]$ are $\partial^2 y / \partial \theta_k \partial \theta_m$.

5.3 Pass-through of unmodelled inputs

With `attach.input = TRUE`, inputs that do not appear among the variables and parameters of (51) are appended to the output unchanged: their values are copied, their Jacobian rows are zero for a variable and the identity for a parameter, and their Hessian rows are zero. An observation map then need not repeat every column of the integrator output as an identity, as with the partial-shape **tangent** of `solveODE()`.

6 Benchmarks

Both backends take the right-hand side, the Jacobian, the events and the linear-solver choice from the same expression graph and run at the same tolerances, so a head-to-head measurement compares the numerical cores.

Setup. The suite comprises 43 problems: 33 published systems-biology models from the PETab benchmark collection and 10 classical test problems (van der Pol at $\mu = 1000$, Robertson, OREGO, HIRES, E5, Pollution, and the one-dimensional Brusselator and FitzHugh-Nagumo systems at $N = 24$ and 64), with $n_x = 2$ to 504 states. First-order sensitivities run forward for up to $M = 32$ parameters and in reverse from $M = 120$ on, which concerns Lang_PLOSCoMputBiol2024 ($n_x = 124$, $M = 164$) and Chen_MSB2009 ($n_x = 504$, $M = 152$). Both backends use BDF/NDF with the automatically selected linear solver at $(\text{rtol}, \text{atol}) \in \{(10^{-4}, 10^{-6}), (10^{-7}, 10^{-9}), (10^{-10}, 10^{-12})\}$, and a cell reports the median of five batches of at least 0.25 s. The suite ran on AMD EPYC 7513 nodes with twelve workers each; both backends of a cell ran in the same worker. Ratios are geometric means over the three tolerances; Figure 1 shows $t_{\text{cvoid}}()/t_{\text{cppode}}()$ per problem.

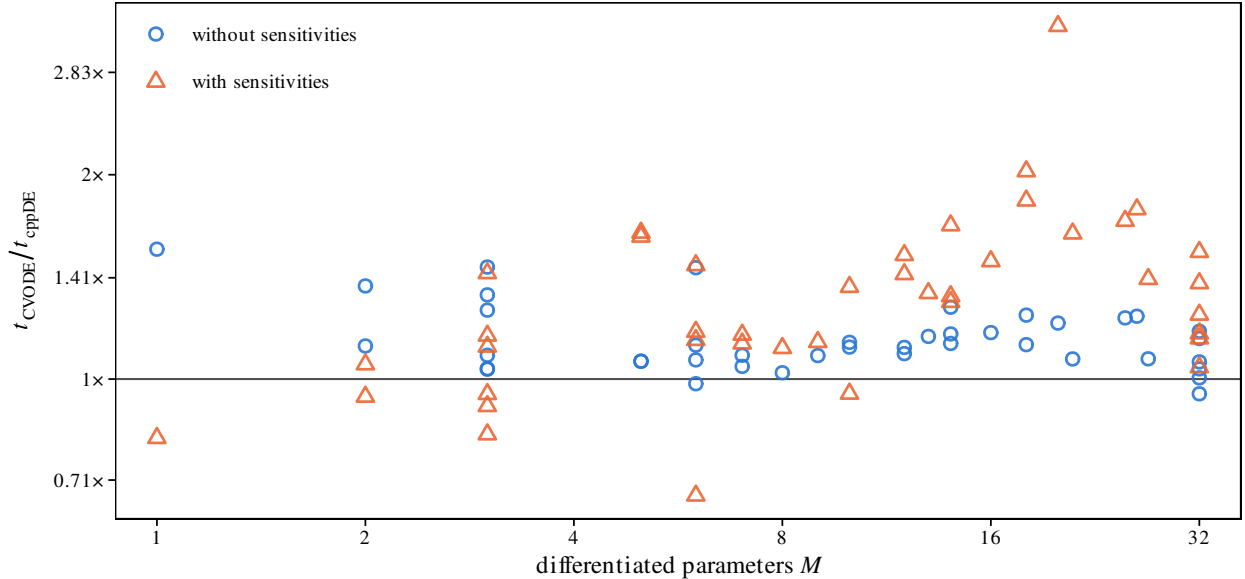


Figure 1: Head-to-head ratio of the two backends, one marker per problem and mode; above the line `cppode()` is faster. The plain solve is placed at the M of its sensitivity counterpart.

Plain solves. `cppode()` is faster on 41 of 43 problems, by a geometric mean of 1.15, and takes the same steps as CVODE: the ratios of accepted steps and right-hand-side evaluations are 1.02 and 1.06. The margin is a cost per step. The model functions are inlined into the stepper, whereas CVODE calls them through function pointers and dispatches every vector operation through the `N_Vector` table.

Forward sensitivities. `cppode()` is faster on 34 of 41 problems, by 1.29, and the ratio grows with M : 1.01 for $M \leq 3$, 1.22 for $4 \leq M \leq 12$ and 1.52 for $13 \leq M \leq 32$. CVODES evaluates one Jacobian-vector product per parameter; the in-tree backend evaluates f once on a dual state that carries all M tangents and needs about one eleventh of the right-hand-side evaluations. The dual evaluation has a cost per state component that does not depend on M , so problems with few parameters fall below parity: six of the seven have $M \leq 6$, the lowest being SalazarCavazos_MBoC2020 ($n_x = 75$, $M = 6$) at 0.67.

Reverse sensitivities. Measured in plain solves, the cost of one gradient by forward sensitivities is linear in M (Figure 2): $1.8 + 0.60M$ for `cppode()` and $1.2 + 0.84M$ for CVODES, about 92 and 129 at $M = 152$. The discrete adjoint of Section 3.9 costs 6.3 plain solves on Chen_MSB2009 and 3.6 on Lang_PLOSCoMputBiol2024, independent of M . The CVODES adjoint of Section 4.3 costs 27 on Lang, 8.8

to 13.7 times the discrete adjoint; Chen carries events, which it does not support.

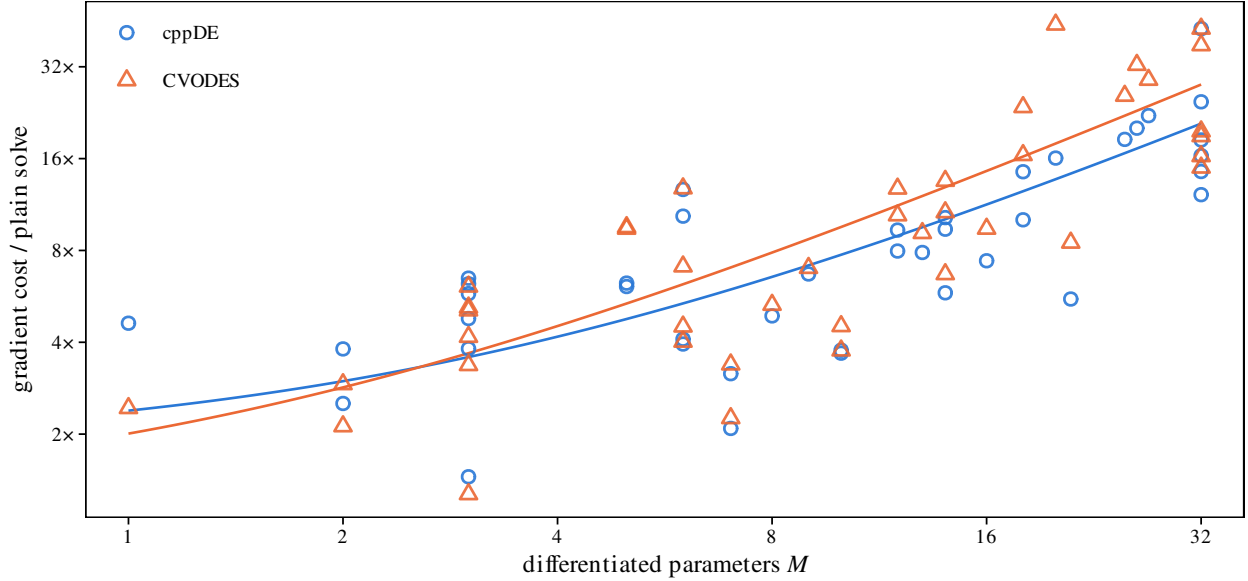


Figure 2: Cost of one gradient by forward sensitivities in units of the plain solve of the same problem and backend, with least-squares lines.

A separate run on the six models with $M \geq 40$, one core per model, compares the two adjoints directly (Figure 3). The CVODES adjoint takes 8.7 to 18.8 times as long on Lang, Lucarelli_CellSystems2018 and Raimundez_PCB2020, and 1.0 to 1.5 times on Zheng_PNAS2012 ($n_x = 15$), where a solve takes about a millisecond. The discrete adjoint runs backward over the steps of the forward solve; CVODES chooses its own backward steps and recomputes the forward state between checkpoints. At $\text{rtol} = 10^{-10}$ both gradients deviate from the reference by 10^{-10} to $5 \cdot 10^{-7}$.

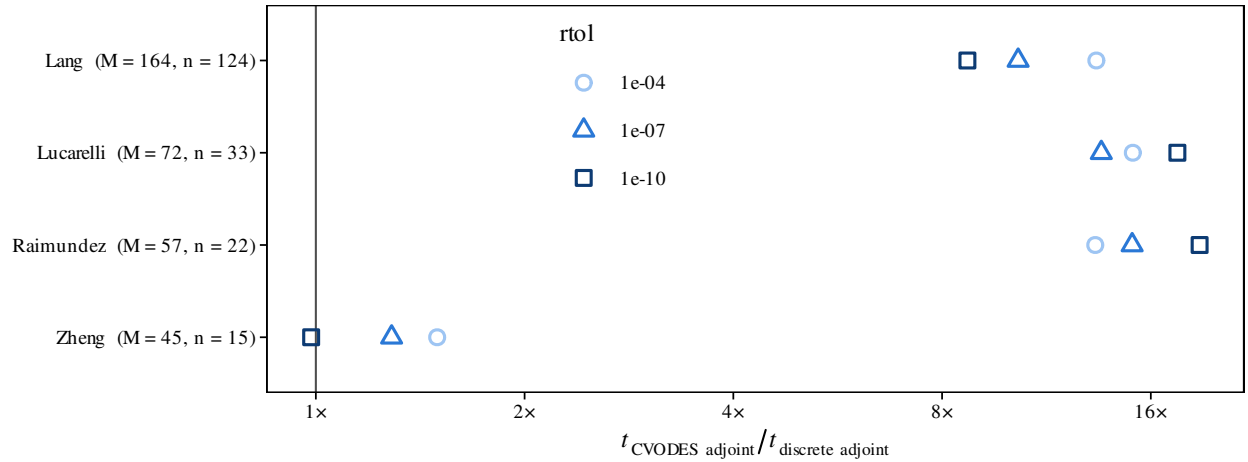


Figure 3: Time of one gradient by the CVODES adjoint over the time by the discrete adjoint, per model and tolerance; right of the line the discrete adjoint is faster.

Second order. The Hessian of the summed outputs is measured for `cppODE()` alone, on models with at most 30 states: forward over reverse (Section 3.9) with up to 32 directions, forward over forward up to 10 states and 10 directions. Fitted in log-log, the cost grows with exponent 1.9 forward over forward and 0.6 forward over reverse (Figure 4). The two break even at $M = 3$; forward over reverse is 3.0 times faster at $M = 6$ and 9

to 14 times at $M = 9$ and 10. At $\text{rtol} = 10^{-10}$ the Hessians deviate from the forward-over-forward Hessian at the reference tolerance by a median of $8.1 \cdot 10^{-10}$ (forward over forward) and $1.8 \cdot 10^{-8}$ (forward over reverse).

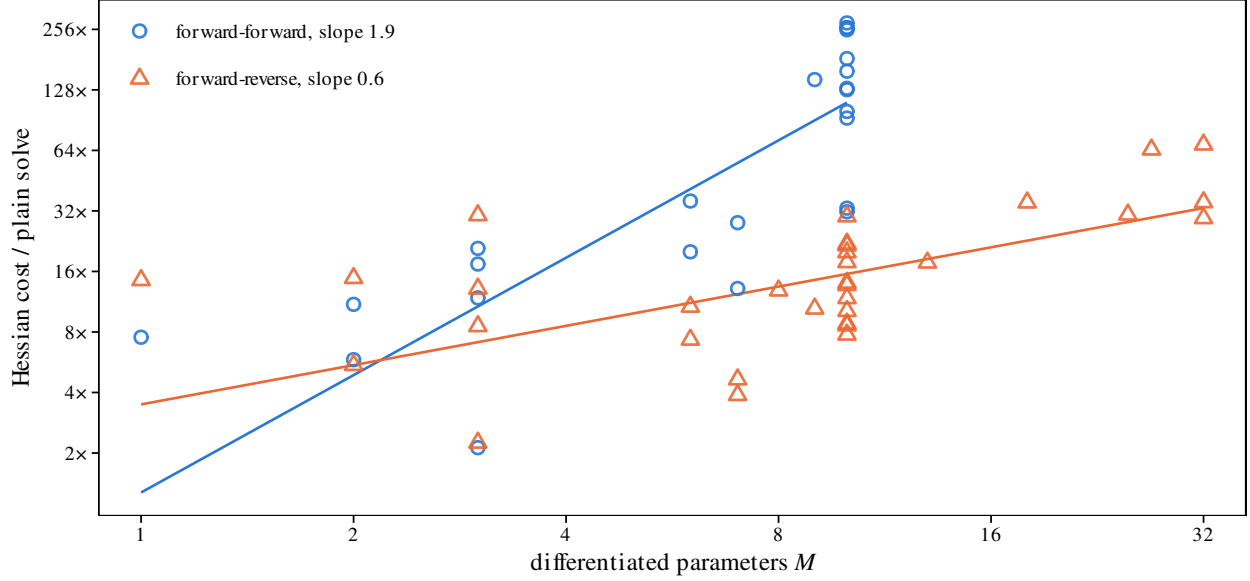


Figure 4: Cost of one Hessian of the summed outputs in units of the plain solve. The lines are power laws fitted in log-log over $M > 1$.

Linear solver and accuracy. Seventeen models were solved again with the linear solver pinned to dense and to sparse. The sparse gain is the same for both backends and grows from 1.09 at $n_x = 8$ to 3.3 at $n_x = 504$ (Figure 5); the automatic selection chose the sparse path in every case. Against a reference at $\text{atol} = 10^{-14}$, $\text{rtol} = 10^{-12}$ the error of `cppODE()` is a median 0.99 times that of CVODE without sensitivities, with the central 90 % of the problems between 0.51 and 1.51, and 1.09 with sensitivities, between 0.66 and 2.37. The reference is itself a CVODE solution, which favours CVODE.

Six of the 1083 cells failed, all at $\text{rtol} = 10^{-10}$: the forward sensitivities of `cppODE()` on Oliveira_NatCom-mun2021 with each linear solver and on Weber_BMC2015, and forward over forward on the same two models.

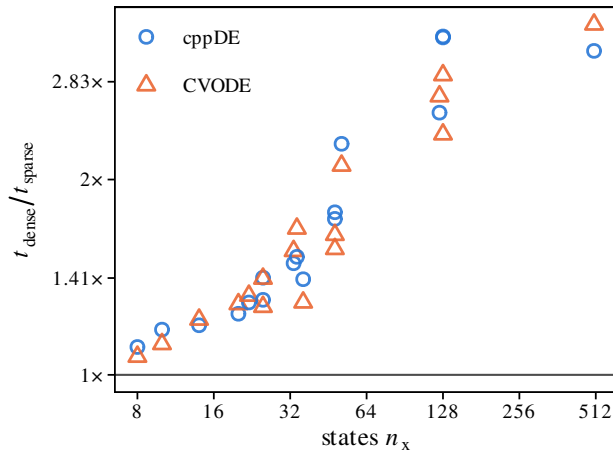


Figure 5: Gain of the sparse over the dense linear solver, one marker per model and backend.

References

- [1] A. C. Hindmarsh *et al.*, “SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 31, no. 3, pp. 363–396, 2005, doi: 10.1145/1089014.1089020.
- [2] L. F. Shampine and M. W. Reichelt, “The MATLAB ODE suite,” *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997, doi: 10.1137/S1064827594276424.
- [3] E. Hairer and G. Wanner, *Solving ordinary differential equations II: Stiff and differential-algebraic problems*. in Springer series in computational mathematics. Springer Berlin Heidelberg, 2010. Available: <https://books.google.de/books?id=zZSv3acps1QC>
- [4] Ch. Tsitouras, “Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption,” *Computers & Mathematics with Applications*, vol. 62, no. 2, pp. 770–775, 2011, doi: 10.1016/j.camwa.2011.06.002.
- [5] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving ordinary differential equations I: Nonstiff problems*. in Springer series in computational mathematics. Springer Berlin Heidelberg, 2008. Available: <https://books.google.de/books?id=F93u7VcSRyYC>
- [6] Y. Cao, S. Li, L. Petzold, and R. Serban, “Adjoint sensitivity analysis for differential-algebraic equations: The adjoint DAE system and its numerical solution,” *SIAM Journal on Scientific Computing*, vol. 24, no. 3, pp. 1076–1089, 2003, doi: 10.1137/S1064827501380630.

A BDF and NDF coefficients

Source. [2], Table 1; cppDE implementation in `cppde_multistepper.hpp`.

The Numerical Differentiation Formula (NDF) of order q modifies the classical BDF q corrector by adding a term proportional to the Nordsieck predictor error,

$$\sum_{m=1}^q \frac{1}{m} \nabla^m x_n - h_n f(x_n, t_n) - \kappa_q \gamma_q (x_n - x_n^{\text{pred}}) = 0, \quad \gamma_q = \sum_{m=1}^q \frac{1}{m}.$$

Setting $\kappa_q = 0$ recovers BDF q . The values κ_q adopted by cppDE (selectable at compile time via the argument `useNDF`, default `TRUE`) are tabulated in Table 6, together with the empirical step-size gain over classical BDF q and the resulting stability angle.

q	κ_q	Step-size gain over BDF q	Stability angle
1	-0.1850	+26 %	A-stable
2	-0.1111	+26 %	A-stable
3	-0.0823	+26 %	80°
4	-0.0415	+12 %	66°
5	0.0000	0 %	51° (= BDF5)

Table 6: NDF correction coefficients κ_q adopted by cppDE, the empirical step-size gain over classical BDF q at matched local error, and the resulting stability angle.

NDF5 is set to $\kappa_5 = 0$ because the BDF5 stability angle of 51° leaves no headroom for further reduction; NDF5 is therefore identical to BDF5.

B Adams-Moulton coefficients

Source. [1], ℓ -vector formulation as in CVODE; cppDE implementation in `adams_set_coefficients()` inside `cppde_multistepper.hpp`.

In contrast to BDF, the Adams-Moulton coefficients depend on the recent step-size history and are recomputed every step from the Nordsieck array. Define $\xi_j = (t_n - t_{n-j})/h_n$; the ℓ -vector recurrence reads

$$\ell_0(q) = 1, \quad \ell_j(q) = \ell_{j-1}(q) + \xi_j^{-1}, \quad j = 1, \dots, q,$$

followed by a polynomial expansion that yields the corrector weights $\beta_0, \dots, \beta_q$ used in the current step. Because the coefficients are step-size adaptive, no static table can reproduce them in general; the canonical fixed-step values for $h_n = h_{n-1} = \dots$ are given in standard references [5], Table III.1.1 and reduce to the recurrence above in that limit.

The maximum order in cppDE is $q_{\max} = 12$, matching CVODE's default; the order controller may select any $q \in \{1, \dots, 12\}$ at run time.

C Rosenbrock4 coefficients

Source. [3, Sec. IV.7]; identical to the default coefficients in the `rosenbrock4` stepper of Boost.Numeric.Odeint. cppDE implementation in `default_rosenbrock_coefficients` in `cppde_rosenbrock4.hpp`.

A Rosenbrock method of s stages with diagonal coefficient γ is defined, with iteration matrix $W = (\gamma h_n)^{-1} I -$

$J(x_n, t_n)$, by the stage equations

$$W k_i = f\left(x_n + \sum_{j<i} \alpha_{ij} k_j, t_n + c_i h_n\right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma} k_j + h_n d_i \frac{\partial f}{\partial t},$$

$$x_{n+1} = x_n + \sum_{i=1}^s m_i k_i, \quad \hat{x}_{n+1} = x_n + \sum_{i=1}^s \hat{m}_i k_i.$$

The cppDE parameterisation uses $s = 6$ stages with $\gamma = 1/4$. The numerical values of all coefficients are reproduced in the following tables, indexed exactly as in the source code: the diagonal coefficient and the time partials and node positions in Table 7, the stage couplings α_{ij} and γ_{ij} in Tables 8 and 9, the sixth-row weights in Table 10, and the dense-output coefficients in Table 11.

C.1 Diagonal coefficient, time partials, and node positions

γ	0.25	d_1	0.25	c_2	0.386
d_2	-0.1043	d_3	0.1035	c_3	0.21
d_4	0.0362			c_4	0.63

Table 7: Rosenbrock4 diagonal coefficient γ , time-partial coefficients d_i , and stage node positions c_i .

C.2 Stage couplings α_{ij}

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$\alpha_{2,j}$	1.5440000000			
$\alpha_{3,j}$	0.9466785281	0.2557011699		
$\alpha_{4,j}$	3.3148251871	2.8961240160	0.9986419140	
$\alpha_{5,j}$	1.2212245092	6.0191344813	12.5370833293	-0.6878860361

Table 8: Rosenbrock4 stage-coupling coefficients α_{ij} . Row index i is the stage being formed; column index j ranges over earlier stages.

C.3 Stage couplings γ_{ij}

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$\gamma_{2,j}$	-5.6688000000			
$\gamma_{3,j}$	-2.4300933568	-0.2063599157		
$\gamma_{4,j}$	-0.1073529058	-9.5945622510	-20.4702861481	
$\gamma_{5,j}$	7.4964433140	-10.2468043146	-33.9999035282	11.7089089321

Table 9: Rosenbrock4 stage-coupling coefficients γ_{ij} , appearing in the $(1/h_n) \sum_{j<i} (\gamma_{ij}/\gamma) k_j$ term of the stage equation. Row and column conventions match Table 8.

C.4 Solution and error weights

The advanced solution and the embedded error estimate are formed from the stage values $k_1, \dots, k_5$ via

$$x_{n+1} = x_n + \sum_{i=1}^5 \alpha_{6,i} k_i, \quad e_{n+1} = \sum_{i=1}^5 \gamma_{6,i} k_i,$$

with the sixth-row coefficients listed in Table 10.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$\alpha_{6,i}$	1.2212245092	6.0191344813	12.5370833293	-0.6878860361	1.0
$\gamma_{6,i}$	8.0832467959	-7.9811329881	-31.5215943287	16.3193054312	-6.0588182388

Table 10: Rosenbrock4 sixth-row coefficients. The $\alpha_{6,i}$ values combine the stage values into the advanced solution x_{n+1} ; the $\gamma_{6,i}$ values supply the embedded order-3 error estimate e_{n+1} .

C.5 Dense-output coefficients

Continuous output on $[t_n, t_{n+1}]$ is constructed from the five intermediate stage values together with two auxiliary coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$, listed in Table 11, that combine the stages with the step solution to produce an order-3 interpolant.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$d_i^{(2)}$	10.1262350834	-7.4879958776	-34.8009186156	-7.9927717076	1.0251377233
$d_i^{(3)}$	-0.6762803393	6.0877146517	16.4308432089	24.7672251142	-6.5943891257

Table 11: Rosenbrock4 dense-output coefficients $d_i^{(2)}$ and $d_i^{(3)}$. Together with the stage values they form the order-3 interpolant on $[t_n, t_{n+1}]$ used to serve user output points falling between accepted steps.

D Tsitouras 5(4) coefficients

Source. [4], Table 1 and equations (7) and (8); cppDE implementation in `cppde_tsit5.hpp`.

The Butcher tableau of the Tsitouras 5(4) pair, reproduced in Table 12, has $s = 7$ stages and satisfies the first column simplifying assumption $\sum_i b_i a_{ij} = b_j(1 - c_j)$ for every j , but not the row condition $\sum_j a_{ij} c_j = c_i^2/2$; the seventh stage is the FSAL stage. Both rows of weights are listed beneath the A matrix: b_i for the order-5 solution and $\hat{b}_i$ for the embedded order-4 estimate.

0							
0.161	0.161						
0.327	-0.0084806555	0.3354806555					
0.9	2.8971530571	-6.3594484900	4.3622954329				
0.9800255409	5.3258648284	-11.7488835641	7.4955393429	-0.0924950664			
1.0	5.8614554429	-12.9209693178	8.1593678986	-0.0715849733	-0.0282690504		
1.0	0.0964607668	0.01	0.4798896504	1.3790085741	-3.2900695154	2.3247105241	
b_i	0.0964607668	0.01	0.4798896504	1.3790085741	-3.2900695154	2.3247105241	0
$\hat{b}_i$	0.0946807557	0.0091835655	0.4877705284	1.2342975669	-2.7077123499	1.8666284182	0.0151515152

Table 12: Butcher tableau of the Tsitouras 5(4) pair. The first column lists the node positions c_i ; the centre block is the strictly lower-triangular coefficient matrix a_{ij} . The b_i row gives the order-5 weights, and the $\hat{b}_i$ row gives the embedded order-4 weights.

The seventh stage is $k_7 = f(x_{n+1}, t_n + h_n)$, which is re-used as k_1 at step $n + 1$. Because $b_7 = 0$, the order-5 solution does not depend on k_7 ; because $\hat{b}_7 \neq 0$, the embedded order-4 estimate does. The local-error estimator used by the step controller is

$$e_n = h_n \sum_{i=1}^7 (\hat{b}_i - b_i) k_i.$$

The seven differences $\hat{b}_i - b_i$ are stored in `cppde_tsit5.hpp` as the coefficients `e1`, ..., `e7` and reproduced in Table 13.

i	$\hat{b}_i - b_i$
1	-0.0017800111
2	-0.0008164345
3	0.0078808780
4	-0.1447110072
5	0.5823571655
6	-0.4580821059
7	0.0151515152

Table 13: Tsit5 embedded error coefficients $\hat{b}_i - b_i$, stored in `cppde_tsit5.hpp` as `e1`, ..., `e7`. The seven values sum to numerical zero, the consistency condition for the embedded pair.

The seven values sum to numerical zero, as required for the consistency of the embedded pair, and the row $a_{7,j}$ of the A matrix coincides with b_j for $j = 1, \dots, 6$, which is the FSAL property.